

采用多种协议。IETF 还综合了许多公司的类似技术, 如思科公司的**标签交换(Tag Switching)**, 以及 Ipsilon 公司的**IP 交换(IP Switching)**等。2001 年 1 月 MPLS 终于成为互联网的建议标准 [RFC 3031, 3032]。

MPLS 利用面向连接技术, 使每个分组携带一个叫作**标签(label)**^①的小整数(这叫作打上标签)。当分组到达交换机(即标签交换路由器)时, 交换机读取分组的标签, 并用标签值来检索分组转发表。这样就比查找路由表来转发分组要快得多。

人们经常把 MPLS 与**异步传递方式 ATM (Asynchronous Transfer Mode)**联系起来, 这仅仅是因为它们都采用了面向连接的工作方式。以前很多人都曾认为网络的发展方向是以 ATM 为核心的宽带综合业务数字 B-ISDN。然而价格低廉得多的高速 IP 路由器仍然占领了市场, 最终导致 ATM 技术和 B-ISDN 未能够成为网络的发展方向。MPLS 并没有取代 IP, 而是作为一种 IP 增强技术, 被广泛地应用在互联网中。

MPLS 具有以下三个方面的特点: (1) 支持面向连接的服务质量。(2) 支持流量工程, 均衡网络负载。(3) 有效地支持虚拟专用网 VPN。

下面讨论 MPLS 的基本工作原理。

4.9.1 MPLS 的工作原理

1. 基本工作过程

在传统的 IP 网络中, 分组每到达一个路由器, 都必须查找转发表, 并按照“最长前缀匹配”的原则找到下一跳的 IP 地址(请注意, 前缀的长度是不确定的)。当网络很大时, 查找含有大量项目的转发表要花费很多的时间。在出现突发性的通信量时, 往往还会使缓存溢出, 这就会引起分组丢失、传输时延增大和服务质量下降。

MPLS 的一个重要特点就是在 MPLS 域的入口处, 给每一个 IP 数据报打上固定长度“**标签**”, 然后对打上标签的 IP 数据报用硬件进行转发, 这就使得 IP 数据报转发的过程大大地加快了^②。采用硬件技术对打上标签的 IP 数据报进行转发就称为**标签交换**。“交换”也表示在转发时不再上升到第三层查找转发表, 而是**根据标签在第二层(链路层)用硬件进行转发**。MPLS 可使用多种链路层协议, 如 PPP、以太网、ATM 以及帧中继等。图 4-66 是 MPLS 协议的基本原理的示意图。

MPLS 域(MPLS domain)是指该域中有许多彼此相邻的路由器, 并且所有的路由器都是支持 MPLS 技术的**标签交换路由器 LSR (Label Switching Router)**。LSR 同时具有标签交换和路由选择这两种功能, 标签交换功能是为了快速转发, 但在这之前 LSR 需要使用路由选择功能构造转发表。



① 注: label 的标准译名本来是“标号”[MINGCI94], 但此译名未被普遍接受。本书曾采用译名“标记”, 现在改用多数文献中的译名“标签”。

② 注: 有的公司愿意用“第三层交换”表示“第三层的路由选择功能加上第二层的交换功能”。但这样的术语不够明确, 因而编者并不主张使用这一术语。

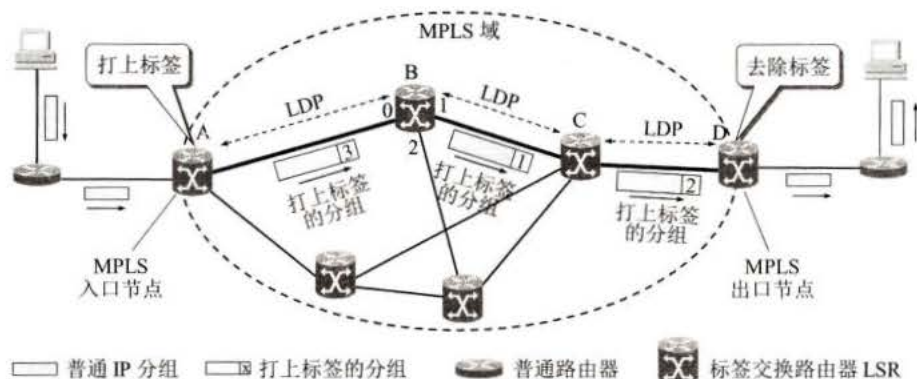


图 4-66 MPLS 协议的基本原理

图 4-66 中给出了 MPLS 的基本工作过程，解释如下：

(1) MPLS 域中的各 LSR 使用专门的标签分配协议 LDP (Label Distribution Protocol) 交换报文，并找出和特定标签相对应的路径，即标签交换路径 LSP (Label Switched Path)。例如在图中的路径 A→B→C→D。各 LSR 根据这些路径构造出转发表。这个过程和路由器构造自己的路由表相似[RFC 3031]，限于篇幅，这里不讨论转发表构造的详细步骤。但应注意的是，MPLS 是面向连接的，因为在标签交换路径 LSP 上的第一个 LSR 就根据 IP 数据报的初始标签确定了整个的标签交换路径，就像一条虚连接一样。

(2) 当一个 IP 数据报进入到 MPLS 域时，MPLS 入口节点(ingress node)就给它打上标签（后面我们会知道，这实际上是插入一个 MPLS 首部），并按照转发表把它转发给下一个 LSR。以后的所有 LSR 都按照标签进行转发。

给 IP 数据报打标签的过程叫作分类(classification)。严格的第三层（网络层）分类只使用了 IP 首部中的字段，如源 IP 地址和目的 IP 地址等。大多数运营商实现了第四层（运输层）分类（除了要检查 IP 首部外，运输层还要检查 TCP 或 UDP 首部中的协议端口号），而有些运营商则实现了第五层（应用层）分类（更进一步地检查数据报的内部并考虑其有效载荷）。

(3) 由于在全网内统一分配全局标签数值是非常困难的，因此一个标签仅仅在两个标签交换路由器 LSR 之间才有意义。分组每经过一个 LSR，LSR 就要做两件事：一是转发，二是更换新的标签，即把入标签更换成为出标签。这就叫作标签对换(label swapping)^①。做这两件事所需的数据都已清楚地写在转发表中。例如，图 4-66 中的标签交换路由器 B 从入接口 0 收到一个入标签为 3 的 IP 数据报，查找了如下的转发表：

入接口	入标签	出接口	出标签
0	3	1	1

标签交换路由器 B 就知道应当把该 IP 数据报从出接口 1 转发出去，同时把标签对换为 1。

当 IP 数据报进入下一个 LSR 时，这时的入标签就是刚才得到的出标签。因此，标签交换路由器 C 接着在转发该 IP 数据报时，又把入标签 1 对换为出标签 2。

^① 注：这里使用[RFC 3031]中的标准词汇。“对换”和“交换”的意思相近，但“对换”更强调两个标签互相对换（把入标签更换为出标签）。虽然 MPLS 中的 LS 是表示“标签交换”，但标签交换路由器 LSR 实现的功能是“标签对换”。

(4) 当 IP 数据报离开 MPLS 域时, MPLS 出口节点(egress node)就把 MPLS 的标签去除, 把 IP 数据报交付非 MPLS 的主机或路由器, 以后就按照普通的转发方法进行转发。

上述的这种“由入口 LSR 确定进入 MPLS 域以后的转发路径”称为**显式路由选择**(explicit routing), 它和互联网中通常使用的“每一个路由器逐跳进行路由选择”有着很大的区别。

下面再讨论 MPLS 中的几个重要概念。

2. 转发等价类 FEC

MPLS 有个很重要的概念就是**转发等价类 FEC** (Forwarding Equivalence Class)。所谓“转发等价类”就是路由器**按照同样方式对待**的 IP 数据报的集合。这里“按照同样方式对待”表示从同样接口转发到同样的下一跳地址, 并且具有同样服务类别和同样丢弃优先级等。FEC 的例子是:

(1) 目的 IP 地址与某一个特定 IP 地址的前缀匹配的 IP 数据报 (这就相当于普通的 IP 路由器);

(2) 所有源地址与目的地址都相同的 IP 数据报;

(3) 具有某种服务质量需求的 IP 数据报。

总之, 划分 FEC 的方法不受什么限制, 这都由网络管理员来控制, 因此非常灵活。入口节点并不是给每一个 IP 数据报指派一个不同的标签, 而是将属于同样 FEC 的 IP 数据报都指派同样的标签。FEC 和标签是一一对应的关系。

图 4-67 给出一个把 FEC 用于负载均衡的例子。图 4-67(a)的主机 H_1 和 H_2 分别向 H_3 和 H_4 发送大量数据。路由器 A 和 C 是数据传输必须经过的。但传统的路由选择协议只能选择最短路径 $A \rightarrow B \rightarrow C$, 这就可能导致这段最短路径过载。

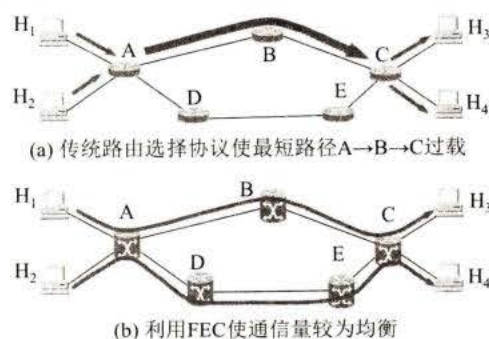


图 4-67 FEC 用于负载均衡

图 4-67(b)表示在 MPLS 的情况下, 入口节点 A 可设置两种 FEC: “源地址为 H_1 而目的地址为 H_3 ”和“源地址为 H_2 而目的地址为 H_4 ”, 把前一种 FEC 的路径设置为 $H_1 \rightarrow A \rightarrow B \rightarrow C \rightarrow H_3$, 而后一种的路径设置为 $H_2 \rightarrow A \rightarrow D \rightarrow E \rightarrow C \rightarrow H_4$ 。这样可使网络的负载较为均衡。网络管理员采用自定义的 FEC 就可以更好地管理网络的资源。这种均衡网络负载的做法也称为**流量工程 TE** (Traffic Engineering)^①或**通信量工程**。

^① 注: 流量工程是对网络上的通信量进行测量、建模和控制, 使网络运行的性能得到最优化。

4.9.2 MPLS 首部的位置与格式

MPLS 并不要求下层的网络都使用面向连接的技术。因此一对 MPLS 路由器之间的物理连接,既可以由一个专用电路组成,如 OC-48 线路,也可以使用像以太网这样的网络。但是这些网络并不提供打标签的手段,而 IPv4 数据报首部也没有多余的位置存放 MPLS 标签。这就需要使用一种封装技术:在把 IP 数据报封装成以太网帧之前,先要插入一个 MPLS 首部。从层次的角度看,MPLS 首部就处在第二层和第三层之间(如图 4-68 所示)。在把加上 MPLS 首部的 IP 数据报封装成以太网帧时,以太网类型字段在单播的情况下设置为 8847_{16} (下标 16 表示这是十六进制的数字),而在多播的情况下为 8848_{16} 。这样,接收方可以用帧的类型来判决这个帧是携带了 MPLS 标签还是一个常规的 IP 数据报。

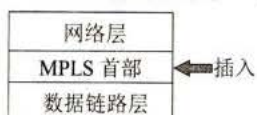


图 4-68 MPLS 首部的位置

图 4-69 给出了 MPLS 首部的格式。可见“给 IP 数据报打上标签”其实就是在以太网的帧首部和 IP 数据报的首部之间插入一个 4 字节的 MPLS 首部。具体的标签就在“标签值”这个字段中。

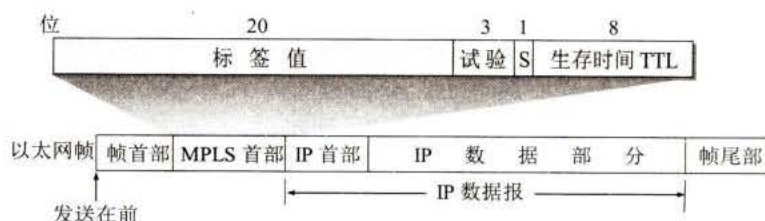


图 4-69 MPLS 首部的格式

MPLS 首部共包括以下四个字段:

(1) **标签值** 占 20 位。由于一个 MPLS 标签占 20 位,因此从理论上讲,在设置 MPLS 时可以使用标签的所有 20 位,因而可以同时容纳高达 2^{20} 个流(即 1048576 个流)。但是,实际上几乎没有哪个 MPLS 实例会使用很大数目的流,因为通常需要管理员人工管理和设置每条交换路径。

(2) **试验** 占 3 位,目前保留用于试验。

(3) **S** 占 1 位, S (Stack)表示栈,在有“标签栈”时使用。

(4) **生存时间 TTL** 占 8 位,用来防止 MPLS 分组在 MPLS 域中兜圈子。

4.9.3 新一代的 MPLS

MPLS 问世后就在互联网中得到了大量的部署。虽然 MPLS 能够更快地转发分组,但其有关控制协议(如 LDP)却比较复杂,其扩展性差,运行维护也较困难。协议 LDP 也无法做到基于时延或带宽等要求的流量调度。为了根据需要灵活地选择流量的转发路径,就还需要再使用资源预留协议 RSVP (见 8.4.3 节)。但 RSVP 的信令非常复杂,每个节点都要维护一个庞大的链路信息数据库。此外,RSVP 不支持等价多路径路由选择 ECMP (Equal-Cost Multipath Routing),而只会选择一条最优路径进行转发。

为了解决上述问题,一种保留了 MPLS 的主要特点,但更加简单的新的源路由选择协议出现了。这就是**段路由选择协议(Segment Routing)**,简称为 SR。但不少人都使用“段路由协议”这样更加简单的名称。在 SR 中,“段(segment)”就是标签,也就是转发指令的一种标识符。SR 的工作原理仍然是基于标签交换的,不过现在不需要使用协议 LDP,因此简化了设备运行的协议数量。SR 由源节点为发送的报文指定路径,并将路径转换成有序的段列表(Segment List),即 MPLS 标签栈,它被封装在分组首部。网络中的其他节点就执行首部中的指令(即标签)进行转发。

整个网络设有控制器,也就是 SDN 控制器(见 4.10 节)。控制器收集并掌握全网的拓扑信息和链路状态信息,计算出分组应传送的整个路径。控制器负责给分组分配 SR 标签,这些标签指明了分组从源点到终点的路径。当分组到达某个网络节点时,节点就根据分组携带的标签转发到下一个节点。

现在 SR 还向 IPv6 演进,这就是 SRv6。它直接利用 IPv6 字段作为标签寻址。而前面介绍的在 MPLS 基础上的 SR 则称为 SR-MPLS。

4.10 软件定义网络 SDN 简介

SDN 的概念最初由斯坦福大学 N. McKeown 于 2009 年首先提出。当时还只是在学术界进行探讨的一种新的网络体系结构。但随后几年发展很快,不少企业相继采用。其中最成功的案例就是谷歌建于 2010 ~ 2012 年的数据中心网络 B4。几年来网络 B4 运行的结果证明,基于 SDN 的专用广域网确实可以大大提高网络带宽利用率,网络运行更加稳定,管理更加高效简化,运行费用也明显降低了。目前,SDN 已经引起人们的密切关注。

在本章开始的 4.1.2 节,我们初步介绍了网络层数据层面和控制层面的基本概念。现在有关 SDN 的资料已相当丰富。限于篇幅,下面我们只能对 SDN 进行简单的介绍。

我们知道,在 SDN 中,数据层面中的交换机是由控制层面进行控制的,图 4-70 表明这种控制是通过协议 OpenFlow 来实现的。协议 OpenFlow 是一个得到高度认可的标准,在讨论 SDN 时往往与 OpenFlow 一起讨论。因此,有人会误认为 SDN 就是 OpenFlow。其实二者有着很大的区别。SDN 不是协议,更不是一种产品。SDN 是一个体系结构,是一种设计、构建和管理网络的新方法或新概念,其要点就是把网络的控制层面和数据层面分离,而让控制层面利用软件来控制数据层面中的许多设备。可以把协议 OpenFlow 看成是在 SDN 体系结构中控制层面和数据层面之间的通信接口,它使得控制层面的控制器可以对数据层面的物理设备或虚拟设备,进行直接访问和操纵。这种控制在逻辑上是集中式的,是基于流的控制。

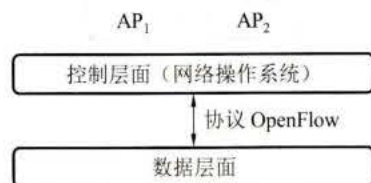


图 4-70 协议 OpenFlow 是控制层面和数据层面的接口

协议 OpenFlow 1.0 是 2009 年底发表的最早的版本,之后每年都进行更新,经过 12 次

更新，到 2015 年 3 月发布了版本 1.5.1 (283 页)，但目前较为成熟的是 1.3 版本。OpenFlow 的技术规范由开放网络基金会 ONF (Open Networking Foundation) 负责制定。这是一个非营利性的产业联盟，其任务是致力于 SDN 的发展和标准化。但请注意，SDN 并未规定必须使用 OpenFlow，只不过是大部分 SDN 的产品采用了 OpenFlow 作为其控制层面与数据层面的接口。

传统意义上的数据层面的任务就是根据转发表来转发分组。可以再把“转发”细分一下。实际上这里有两个步骤。第一个步骤是“匹配”，即查找转发表中的网络前缀，进行最长前缀匹配。第二个步骤是“动作”，即把分组从指明的接口转发出去。这种“匹配 + 动作”的转发方式在 SDN 中得到了扩充，增加了新的内容，变成了广义的转发。这种广义的转发使得“匹配 + 动作”有了新的内容。

在 SDN 的广义转发中，“匹配”能够对不同层次（链路层、网络层、运输层）的首部中的字段进行匹配，而“动作”则不仅是转发分组，而且可以把具有同样目的地址的分组从不同的接口转发出去（为了负载均衡）。还可以重写 IP 首部（如同在 NAT 路由器中的地址转换），或者可以人为地阻挡或丢弃一些分组（如同在防火墙中一样，见 7.7.1 节）。请注意，这里为了讨论问题的方便，在讨论 SDN 的问题时，不管在哪一层传送的数据单元，都称为分组。

这样，在 SDN 的广义转发中，这种完成“匹配 + 动作”的设备，就不应当称为路由器了，而是叫作“分组交换机”或“OpenFlow 交换机”，或更简单些就称为“交换机”。这种交换机并不局限在网络层工作（例如，可使用 L2/L3 交换机）。在 SDN 中，取代传统转发表的是“流表” (flow table)。因此，流表就是“匹配 + 动作”的转发表。

图 4-71 强调了一个重要概念：OpenFlow 交换机中的流表是由远程控制器来管理的，而远程控制器通过一个安全信道（见 7.6.2 节），使用 OpenFlow 协议来管理 OpenFlow 交换机中的流表。这样，OpenFlow 就有了双重意义。一方面，OpenFlow 是 SDN 远程控制器与网络设备之间的通信协议；另一方面，OpenFlow 又是网络交换功能的逻辑结构的规约。我们还应注意到，尽管网络设备可以由不同厂商来生产，同时也可以使用在不同类型的网络中，但从 SDN 远程控制器看到的，则是统一的逻辑交换功能。



图 4-71 OpenFlow 协议与 OpenFlow 交换机

我们在 4.5.1 节介绍 IPv6 首部的流标号时，曾初步地提到“流”的概念。在 OpenFlow 的各种文档中都没有“流”的定义。其实从 OpenFlow 交换机的角度来看，一个流就是穿过网络的一种分组序列，而在此序列中的分组都共享分组首部某些字段的值。例如，某个流可以是具有相同源 IP 地址和目的 IP 地址的所有分组。

图 4-72 给出了 OpenFlow 1.0 版本的流表和分组的首部匹配字段（这是最简单的一个版本，便于用来讲解工作原理）[KUROI17]。每个 OpenFlow 交换机必须有一个或一个以上的流表。每一个流表可以包括很多行，即多个流表项 (flow entry)，它包括三个字段，即首部字

段值（又称为匹配字段）、计数器和动作。下面解释这三个字段的意思。



图 4-72 OpenFlow 1.0 版本的流表和分组的首部匹配字段

首部字段值：这是一组字段，用来使入分组(incoming packet)的对应首部与之相匹配，因此又称为**匹配字段**。匹配不上的分组就被丢弃，或发送到远程控制器做更多的处理。图 4-72 所示的匹配字段有 11 个项目涉及三个层次的首部。这就是说，OpenFlow 的匹配抽象与我们以前讲过的分层的原则明显不同。在 OpenFlow 交换机中，既可以处理链路层的帧，也可以处理网络层的 IP 分组和运输层的报文。

计数器：这是一组计数器，可包括已经与该表项匹配的分组数量，以及从该表项上次更新到现在经历的时间。

动作：这是一组动作，例如，当分组匹配某个流表项时把分组转发到指定的端口，或丢弃该分组，或把分组进行复制后再从多个端口转发出去，或重写分组的首部字段（第二、三和四层的首部字段）。

为了更好地理解流表的匹配与动作，我们讨论下面几个例子[KURO17]。图 4-73 给出的简单网络有 6 台主机（ $H_1 \sim H_6$ ），其 IP 地址标注在主机旁边，还有 3 台分组交换机（ $S_1 \sim S_3$ ）。每台交换机有 4 个端口（即接口，编号为 1 至 4）。还有 1 台 OpenFlow 控制器来控制这些分组交换机的“匹配 + 动作”。

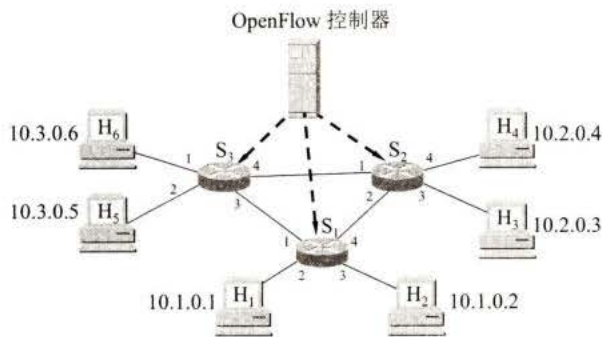


图 4-73 OpenFlow “匹配 + 动作”网络

简单转发的例子

我们设定的转发规则是：来自 H_5 或 H_6 发往 H_3 或 H_4 的分组，都先从 S_3 转发到 S_1 ，然后再从 S_1 转发到 S_2 ，但不通过 S_3 到 S_2 的链路。根据这个转发规则，可以得出交换机 S_3 的流表项是：

匹配	动作
IP 源地址 = 10.3.*.*; IP 目的地址 = 10.2.*.*	转发(3)

这里使用了通配符*。例如，地址 10.3.*.*，表明这样的地址将匹配前 16 位为 10.3 的任何地址。“转发(3)”表明分组转发出去端口是交换机编号为 3 的端口。

交换机 S₁ 的流表项（这里和后面都省略了计数器字段）是：

匹配	动作
入端口 = 1; IP 源地址 = 10.3.*.*; IP 目的地址 = 10.2.*.*	转发(4)

和 S₃ 的流表项相比，这里多了“入端口 = 1”。表明“匹配”仅限于从编号 1 的端口进入交换机 S₁ 的分组。

交换机 S₂ 的流表项是：

匹配	动作
IP 源地址 = 10.3.*.*; IP 目的地址 = 10.2.0.3	转发(3)
IP 源地址 = 10.3.*.*; IP 目的地址 = 10.2.0.4	转发(4)
.....	

负载均衡的例子

在图 4-73 中，为了均衡链路 S₂-S₁ 和链路 S₃-S₁ 的通信量，我们规定：凡是从 H₃ 发往主机 10.1.*.* 的分组，其转发路径应为 S₂→S₁。但凡是 H₄ 发往主机 10.1.*.* 的分组，其转发路径应为 S₂→S₃→S₁。显然，采用基于 IP 目的地址的转发方法，是不能实现这种负载均衡的。但在本例中，只要在交换机 S₂ 的流表项中设置好合适的匹配项目即可。

交换机 S₂ 的流表项是：

匹配	动作
入端口 = 3; IP 目的地址 = 10.1.*.*	转发(2)
入端口 = 4; IP 目的地址 = 10.1.*.*	转发(1)
.....	

防火墙的例子

假定我们在交换机 S₂ 中设置了防火墙，此防火墙的作用是仅仅接收来自与交换机 S₃ 相连的主机所发送的分组（不管是从哪一个端口进来的）。根据这样的规定可得出下列内容。

交换机 S₂ 的流表项是：

匹配	动作
IP 源地址 = 10.3.*.*; IP 目的地址 = 10.2.0.3	转发(3)
IP 源地址 = 10.3.*.*; IP 目的地址 = 10.2.0.4	转发(4)
.....	

虽然上面举出的例子非常简单，但已经可以看出这种广义转发的多样性和灵活性。广义转发的优点是显而易见的。

下面通过图 4-74 简单介绍一下 SDN 的控制层面[KREN15] [KUROI17]。

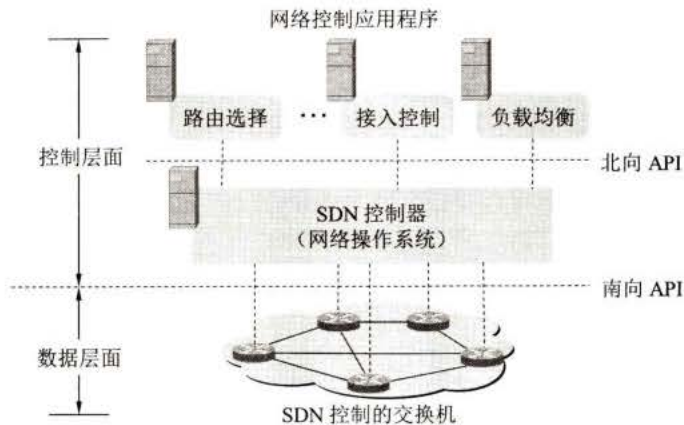


图 4-74 SDN 体系结构的构件

从图 4-74 可以反映出 SDN 体系结构的四个关键特征：

基于流的转发。SDN 控制的交换机分布在数据层面中，而分组的转发可以基于网络层、运输层和链路层协议数据单元中的首部字段值进行。这和传统的路由器仅仅根据 IP 分组的目的地址进行转发，有着很大的区别。SDN 的转发规则都精确规定在交换机中的流表中。所有交换机中的流表项，都是由 SDN 控制器进行计算、管理和安装的。

数据层面与控制层面分离。在许多英语文献中常使用“decouple”一词，相应的中文就是“去耦”。在传统的转发设备路由器中，其数据层面与控制层面都处在同一个设备中，因此二者耦合得非常紧密。但在 SDN 中，则把这两个层面去耦合，使二者不在同一个设备中。这点在图 4-73 中看得很清楚。数据层面有许多相对简单但快速的网络交换机。这些交换机在其流表中执行“匹配 + 动作”的规则。而控制层面则由若干服务器和相应的软件组成，这些服务器和软件决定并管理这些交换机中的流表。

位于数据层面交换机之外的网络控制功能。SDN 中的控制层面是用软件实现的，而且软件是处在不同的机器上，并且可能还远离这些网络交换机。从图 4-74 可以看出，SDN 控制层面包含两个构件，一个是 SDN 控制器（也就是网络操作系统），另一个由若干个网络控制应用程序组成。SDN 控制器维护准确的网络状态信息（例如，远程链路、交换机和主机的状态），把这些信息提供给运行在控制层面的各种控制应用程序，以及提供一些方法，使得这些应用程序能够对底层的许多网络设备进行监视、编程和控制。需要注意的是，在图 4-73 的 SDN 控制器中只画了一个服务器，但这只是强调在逻辑上是集中控制的。实际上，在控制层面中总是使用多个分散的服务器协调地工作，以便实现可扩展性和高可用性。

可编程的网络。通过在控制层面的一些网络控制应用程序，使整个网络成为可编程的。这些应用程序相当于 SDN 控制层面中的“大脑”，SDN 控制器使用这些应用程序，在这些网络设备中指明和控制数据层面。例如，路由选择网络控制应用程序能够确定在源点和终点之间的端到端路径（这需要执行某种算法，也需要使用由 SDN 控制器维护的节点状态和链路状态信息）。网络应用程序还可以进行接入控制，即决定哪些分组在进入某个交换机时必须被阻挡住。此外，网络应用程序在转发分组时还可以执行负载均衡的措施。

从以上的简单例子可以看出，SDN 把网络的许多功能都分散开了。数据层面的交换机、SDN 控制器以及许多网络控制应用程序，这些都可以是分开的实体，并且可以由不同的厂

商和机构来提供。这就和传统网络截然不同。在传统网络中，路由器或交换机是由单独的厂商提供的，其控制层面和数据层面以及协议的实现，都是垂直集成在一个机器里面的。目前出现的这种变化，有点像当初计算机的演变。早期的大型计算机，从硬件到软件以及应用程序，都是由一个单独的厂家生产完成的。但后来演变到现在的个人电脑，其硬件机身、操作系统以及上层的应用程序，可以由多个厂家分别生产和提供，这样的系统就变得更加开放，其功能也更加丰富了。SDN 也可能有这样的发展结果。

图 4-74 还给出了 SDN 控制器和下面数据层面的受控设备的通信接口，即南向 API，以及 SDN 控制器和上面网络控制应用程序的接口，即北向 API。

SDN 控制器是最复杂的，它还可以划分为如图 4-75 所示的三个层次。

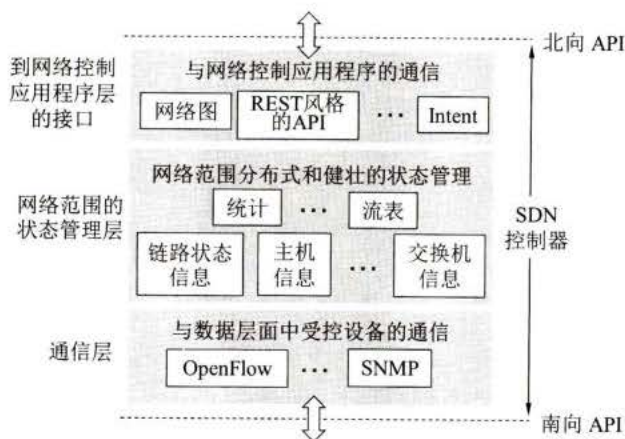


图 4-75 SDN 控制器的三个层次

最下面的一层是**通信层**，其任务是完成 SDN 控制器与受控的网络设备之间的通信。显然，要完成这样的通信，我们必须有一个协议，用来在 SDN 控制器与这些设备之间传送信息。此外，这些设备还必须能够向 SDN 控制器传送在本地观察到的事件（例如，用一个报文指示某条链路正常工作或出了故障断开了，或指示某个设备刚刚接入到网络中，或者某种信号突然出现可以表示某个设备已加电并可以工作）。这样就可保证 SDN 控制器掌握了网络状态的最新视图。在通信层的协议有前面已经提到过的 OpenFlow，以及后面在第 6 章应用层要学习的协议 SNMP，等等。通信层与数据层面的接口叫作**南向 API 接口**。现有用 SDN 控制器概念制作的商品基本上（当然不是全部）都是采用协议 OpenFlow。

在中间的一层是**网络范围的状态管理层**。SDN 控制层面若要做出任何最终的控制决定（例如，在所有的交换机中配置流表以便进行端到端的转发，或实现负载均衡，或实现某种特殊的防火墙能力），就需要让控制器掌握全网的主机、链路、交换机，以及其他受 SDN 控制的设备。交换机的流表中包含有计数器，而网络应用程序需要使用这些计数器的值。因此，这些计数器的值对网络应用程序来说也必须是可用的。由于控制层面的最终目的是确定各种被控设备的流表，因此控制器还需要维护这些流表的副本。所有上述这些信息构成由 SDN 控制器维护的网络范围状态。

最上面一层是**到网络控制应用程序层的接口**。SDN 控制器与网络控制应用程序的交互都要通过北接口。这个 API 接口允许网络控制应用程序对状态管理层里面的网络状态和流表进行读写操作。网络控制应用程序事先已进行了注册。当状态变化的事件出现时，网络控

制应用程序把得到的网络事件进行通告，并采取相应的动作，例如，计算新的最低开销的路径。这一层可以提供不同类型的 API。例如，REST 风格的 API 目前使用得较多。图中的 REST (REpresentational State Transfer)即表述性状态传递，是一种针对网络应用的设计和开发方法[W-REST]。图中的 Intent 是对要进行的操作的一种抽象描述[W-INTENT]，可用它在组件之间传递数据。

目前已经出现了一些开放源代码控制器，或简称为开源控制器(Open Source Controller)，最有代表性的就是 OpenDaylight 和 ONOS。这里就不再进行讨论了。

本章的重要概念

- TCP/IP 体系中的网络层向上只提供简单灵活的、无连接的、尽最大努力交付的数据报服务。网络层不提供服务质量的承诺，不保证分组交付的时限，所传送的分组可能出错、丢失、重复和失序。进程之间通信的可靠性由运输层负责。
- IP 网是虚拟的，因为从网络层上看，IP 网就是一个统一的、抽象的网络（实际上是异构的）。IP 层抽象的互联网屏蔽了下层网络很复杂的细节，使我们能够使用统一的、抽象的 IP 地址处理主机之间的通信问题。
- 互联网上交付主机的方式有两种：在本网络上的直接交付（不经过路由器）和到其他网络的间接交付（经过至少一个路由器，但最后一次一定是直接交付）。
- 一个 IP 地址在整个互联网范围内是唯一的。早期使用的是分类的 IP 地址，包括 A 类、B 类和 C 类地址（单播地址），以及 D 类地址（多播地址）。E 类地址未使用。
- 分类的 IP 地址由网络号字段（指明网络）和主机号字段（指明主机）组成。网络号字段最前面的类别位指明 IP 地址的类别。
- 目前已广泛使用无分类域间路由选择 CIDR 记法把 IP 地址后面加上斜线“/”，斜线后是前缀的位数。前缀（或网络前缀）指明网络，后缀指明主机。前缀相同的连续的 IP 地址构成“CIDR 地址块”。
- CIDR 的 32 位地址掩码（或子网掩码）由一串 1 和一串 0 组成，其中 1 的个数是前缀的长度。只要把 IP 地址和地址掩码按位进行 AND 运算，即可得出网络地址。
- IP 地址是一种分等级的地址结构。IP 地址管理机构在分配 IP 地址时只分配网络前缀（网络号），而主机号则由得到该网络前缀的单位自行分配。路由器仅根据目的主机所连接的网络前缀（网络号）来转发分组。
- IP 地址标志一台主机（或路由器）和一条链路的接口。多归属主机同时连接到两个或更多的网络上。这样的主机同时具有两个或更多的 IP 地址，其网络前缀必须是不同的。由于一个路由器至少应当连接到两个网络，因此一个路由器至少应当有两个不同的 IP 地址。
- 按照互联网的观点，用转发器或网桥连接起来的若干个局域网仍为一个网络。所有分配到网络前缀的网络（不管是范围很小的局域网，还是可能覆盖很大地理范围的广域网）都是平等的。
- MAC 地址（即硬件地址或物理地址）是数据链路层和物理层使用的地址，而 IP 地址是网络层和以上各层使用的地址，是一种逻辑地址（用软件实现的），在数据链路层看不见数据报的 IP 地址。
- IP 数据报分为首部和数据两部分。首部的前一部分是固定长度，共 20 字节，是所

有 IP 数据报必须具有的（源地址、目的地址、总长度等重要字段都在固定首部中）。一些长度可变的可选字段放在固定首部的后面。

- IP 首部中的生存时间字段给出了 IP 数据报在互联网中所能经过的最大路由器数，可防止 IP 数据报在互联网中无限制地兜圈子。
- 地址解析协议 ARP 把 IP 地址解析为 MAC 地址，它解决同一个局域网上的主机或路由器的 IP 地址和 MAC 地址的映射问题。ARP 的高速缓存可以大大减少网络上的通信量。
- 在互联网中，我们无法仅根据 MAC 地址寻找到在某个网络上的某台主机。因此，从 IP 地址到 MAC 地址的解析是非常必要的。
- 路由聚合（把许多前缀相同的地址用一个来代替）有利于减少路由表中的项目，减少路由器之间的路由选择信息的交换，从而提高了整个互联网的性能。
- “转发”和“路由选择”是不同的概念。“转发”是单个路由器的动作。“路由选择”是许多路由器共同协作的过程，这些路由器相互交换信息，目的是生成路由表，再从路由表导出转发表。若采用自适应路由选择算法，则当网络拓扑变化时，路由表和转发表都能够自动更新。在许多情况下，可以不考虑转发表和路由表的区别，而都使用路由表这一名词。
- 自治系统(AS)就是在单一的技术管理下的一组路由器。一个自治系统对其他自治系统表现出的是一个单一的和一致的路由选择策略。
- 路由选择协议有两大类：内部网关协议（或自治系统内部的路由选择协议），如 RIP 和 OSPF；外部网关协议（或自治系统之间的路由选择协议），如 BGP-4。
- RIP 是分布式的基于距离向量的路由选择协议，只适用于小型互联网。RIP 按固定的时间间隔与相邻路由器交换信息。交换的信息是自己当前的路由表，即到达本自治系统中所有网络的（最短）距离，以及到每个网络应经过的下一跳路由器。
- OSPF 是分布式的链路状态协议，适用于大型互联网。OSPF 只在链路状态发生变化时，才向本自治系统中的所有路由器，用洪泛法发送与本路由器相邻的所有路由器的链路状态信息。“链路状态”指明本路由器都和哪些路由器相邻，以及该链路的“度量”。“度量”可表示费用、距离、时延、带宽等，可统称为“代价”。所有的路由器最终都能建立一个全网的拓扑结构图。
- 协议 BGP-4 简称为 BGP，是一种路径向量路由选择协议，用于在不同自治系统 AS 之间的路由选择。BGP 力求在 AS 之间，找出一条能够到达目的网络前缀且较好的路由（不是最佳路由，但不能兜圈子）。各种 BGP 报文是在 AS 之间的 BGP 连接（即 eBGP）和 AS 内部的 BGP 连接（即 iBGP）上传送的。BGP 路由指出，从哪个路由器，经过哪些 AS，就可以到达哪个网络前缀。
- 网际控制报文协议 ICMP 是 IP 层的协议。ICMP 报文作为 IP 数据报的数据，加上首部后组成 IP 数据报发送出去。使用 ICMP 并非为了实现可靠传输。ICMP 允许主机或路由器报告差错情况和提供有关异常情况的报告。ICMP 报文的种类有两种，即 ICMP 差错报告报文和 ICMP 询问报文。
- ICMP 的一个重要应用就是分组网间探测 PING，用来测试两台主机之间的连通性。PING 使用了 ICMP 回送请求与回送回答报文。
- 要解决 IP 地址耗尽的问题，最根本的办法就是采用具有更大地址空间的新版本的

协议 IP，即 IPv6。

- IPv6 所带来的主要变化是：(1) 更大的地址空间（采用 128 位的地址）；(2) 灵活的首部格式；(3) 改进的选项；(4) 支持即插即用；(5) 支持资源的预分配；(6) IPv6 首部改为 8 字节对齐。
- IPv6 数据报在基本首部的后面允许有零个或多个扩展首部，再后面是数据。所有的扩展首部和数据合起来叫作数据报的有效载荷或净负荷。
- IPv6 数据报的目的地址可以是以下三种基本类型地址之一：单播、多播和任播。
- IPv6 的地址使用冒号十六进制记法。
- 向 IPv6 过渡只能采用逐步演进的办法，必须使新安装的 IPv6 系统能够向后兼容。向 IPv6 过渡可以使用双协议栈或使用隧道技术。
- 与单播相比，在一对多的通信中，IP 多播可大大节约网络资源。IP 多播使用 D 类 IP 地址。IP 多播需要使用网际组管理协议 IGMP 和多播路由选择协议。
- 虚拟专用网 VPN 利用公用的互联网作为本机构各专用网之间的通信载体。VPN 内部使用互联网的专用地址。一个 VPN 至少要有有一个路由器具有合法的全球 IP 地址，这样才能和本系统的另一个 VPN 通过互联网进行通信。所有通过互联网传送的数据都必须加密。
- 使用网络地址转换 NAT 技术，可以在专用网络内部使用专用 IP 地址，而仅在连接到互联网的路由器使用全球 IP 地址。这样就大大节约了宝贵的 IP 地址。
- MPLS 的特点：(1) 支持面向连接的服务质量；(2) 支持流量工程，均衡网络负载；(3) 有效地支持虚拟专用网 VPN。
- MPLS 在入口节点给每一个 IP 数据报打上固定长度的“标签”，然后根据标签在第二层（链路层）用硬件进行转发（在标签交换路由器中进行标签对换），因而转发速率大大加快。
- 软件定义网络 SDN 并不是要改变网络的功能，而是一种新的体系结构，其要点是：(1) 基于流的转发；(2) 数据层面与控制层面分离；(3) 控制层面用软件实现，并且是逻辑上的集中式控制；(4) 可编程的网络。
- SDN 控制器有以下三个层次：(1) 通信层，大多采用协议 OpenFlow，与数据层面的接口叫作南向接口。(2) 状态管理层。(3) 到网络控制应用程序层的接口（北向接口）。

习题

- 4-01 网络层向上提供的服务有哪两种？试比较其优缺点。
- 4-02 网络互连有何实际意义？进行网络互连时，有哪些共同的问题需要解决？
- 4-03 作为中间设备，转发器、网桥、路由器和网关有何区别？
- 4-04 试简单说明下列协议的作用：
IP, ARP 和 ICMP。
- 4-05 IP 地址如何表示？
- 4-06 IP 地址的主要特点是什么？
- 4-07 试说明 IP 地址与 MAC 地址的区别。为什么要使用这两种不同的地址？
- 4-08 IP 地址方案与我国的电话号码体制的主要不同点是什么？

- 4-09 IP 数据报中的首部检验和并不检验数据报中的数据。这样做的最大好处是什么？坏处是什么？
- 4-10 当某个路由器发现一 IP 数据报的检验和有差错时，为什么采取丢弃的办法而不是要求源站重传此数据报？计算首部检验和为什么不采用 CRC 检验码？
- 4-11 设 IP 数据报使用固定首部，其各字段的具体数值如图 4-76 所示（除 IP 地址外，均为十进制形式表示）。试用二进制运算方法计算应当写入到首部检验和字段中的数值（用二进制形式表示）。

4	5	0	28	
1			0	0
4	17		首部检验和（待计算后写入）	
10.12.14.5				
12.6.7.9				

图 4-76 习题 4-11 的图

- 4-12 重新计算上题，但使用十六进制运算方法（每 16 位二进制数字转换为 4 个十六进制数字，再按十六进制加法规则计算）。比较这两种方法。
- 4-13 什么是最大传送单元 MTU？它和 IP 数据报首部中的哪个字段有关系？
- 4-14 在互联网中将 IP 数据报分片传送的数据报在最后的目的地主机进行组装。还可以有另一种做法，即数据报分片通过一个网络就进行一次组装。试比较这两种方法的优劣。
- 4-15 一个 3200 位长的 TCP 报文传到 IP 层，加上 160 位的首部后成为数据报。下面的互联网由两个局域网通过路由器连接起来，但第二个局域网所能传送的最长数据帧中的数据部分只有 1200 位，因此数据报在路由器必须进行分片。试问第二个局域网向其上层要传送多少比特的数据（这里的“数据”当然指的是局域网看见的数据）？
- 4-16 (1) 试解释为什么 ARP 高速缓存每存入一个项目就要设置 10~20 分钟的超时计时器。这个时间设置得太大或太小会出现什么问题？
(2) 举出至少两种不需要发送 ARP 请求分组的情况（即不需要请求将某个目的 IP 地址解析为相应的 MAC 地址）。
- 4-17 主机 A 发送 IP 数据报给主机 B，途中经过了 5 个路由器。试问在 IP 数据报的发送过程中总共使用了几次 ARP？
- 4-18 设某路由器建立了如下转发表：

前缀匹配	下一跳
192.4.153.0/26	R ₃
128.96.39.0/25	接口 m0
128.96.39.128/25	接口 m1
128.96.40.0/25	R ₂
192.4.153.0/26	R ₃
*（默认）	R ₄

现共收到 5 个分组，其目的地址分别为：

- (1) 128.96.39.10

- (2) 128.96.40.12
- (3) 128.96.40.151
- (4) 192.4.153.17
- (5) 192.4.153.90

试分别计算其下一跳。

- 4-19** 某单位分配到一个地址块 129.250/16。该单位有 4000 台机器，平均分布在 16 个不同的地点。试给每一个地点分配一个地址块，并算出每个地址块中 IP 地址的最小值和最大值。
- 4-20** 一个数据报长度为 4000 字节（固定首部长度）。现在经过一个网络传送，但此网络能够传送的最大数据长度为 1500 字节。试问应当划分为几个短些的数据报片？各数据报片的数据字段长度、片偏移字段和 MF 标志应为何数值？
- 4-21** 写出互联网的 IP 层查找路由的算法。
- 4-22** 有如下的 4 个 /24 地址块，试进行最大可能的聚合。
- 212.56.132.0/24
212.56.133.0/24
212.56.134.0/24
212.56.135.0/24
- 4-23** 有两个 CIDR 地址块 208.128/11 和 208.130.28/22。是否有哪一个地址块包含了另一个的地址？如果有，请指出，并说明理由。
- 4-24** 已知路由器 R_1 的转发表如表 4-10 所示。

表 4-10 习题 4-24 中路由器 R_1 的转发表

前缀匹配	下一跳地址	路由器接口
140.5.12.64/26	180.15.2.5	m2
130.5.8/24	190.16.6.2	m1
110.71/16	...	m0
180.15/16	...	m2
190.16/16	...	m1
默认	110.71.4.5	m0

试画出各网络和必要的路由器的连接拓扑，标注出必要的 IP 地址和接口。对不能确定的情况应当指明。

- 4-25** 一个自治系统分配到的 IP 地址块为 30.138.118/23，并包含有 5 个局域网，其连接图如图 4-77 所示，每个局域网上的主机数分别标注在图 4-77 上。试给出每一个局域网的地址块（包括前缀）。

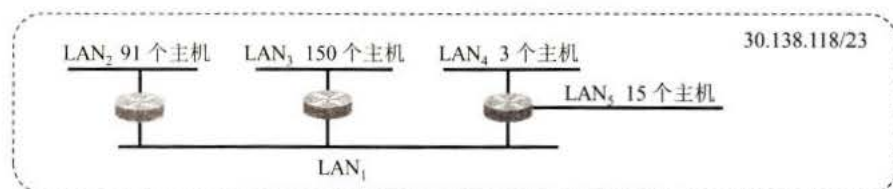


图 4-77 习题 4-25 的图

- 4-26** 一个大公司有一个总部和三个下属部门。公司分配到的网络前缀是 192.77.33/24。公司的网络布局如图 4-78 所示。总部共有 5 个局域网，其中的 LAN₁ ~ LAN₄ 都连接到路由器 R₁ 上，R₁ 再通过 LAN₅ 与路由器 R₂ 相连。R₂ 和远地的三个部门的局域网 LAN₆ ~ LAN₈ 通过广域网相连。每一个局域网旁边标明的数字是局域网上的主机数。试给每一个局域网分配一个合适的网络前缀。

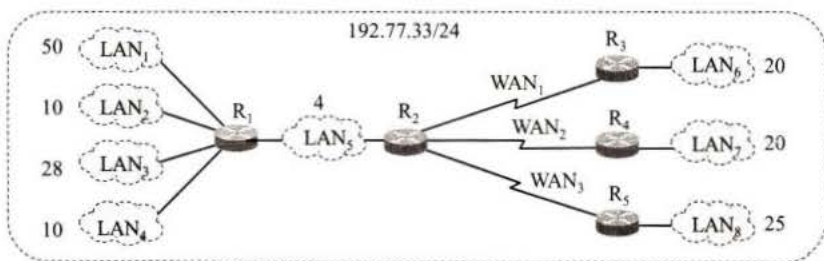


图 4-78 习题 4-26 的图

- 4-27** 以下地址中的哪一个和 86.32/12 匹配？请说明理由。
(1) 86.33.224.123; (2) 86.79.65.216; (3) 86.58.119.74; (4) 86.68.206.154。
- 4-28** 以下的地址前缀中的哪一个地址与 2.52.90.140 匹配？请说明理由。
(1) 0/4; (2) 32/4; (3) 4/6; (4) 80/4。
- 4-29** 下面的前缀中的哪一个和地址 152.7.77.159 及 152.31.47.252 都匹配？请说明理由。
(1) 152.40/13; (2) 153.40/9; (3) 152.64/12; (4) 152.0/11。
- 4-30** 与下列掩码相对应的网络前缀各有多少位？
(1) 192.0.0.0; (2) 240.0.0.0; (3) 255.224.0.0; (4) 255.255.255.252。
- 4-31** 已知地址块中的一个地址是 140.120.84.24/20。试求这个地址块中的最小地址和最大地址。地址掩码是什么？地址块中共有多少个地址？相当于多少个 C 类地址？
- 4-32** 已知地址块中的一个地址是 190.87.140.202/29。重新计算上题。
- 4-33** 某单位分配到一个地址块 136.23.12.64/26。现在需要进一步划分为 4 个一样大的子网。试问：
(1) 每个子网的网络前缀有多长？
(2) 每一个子网中有多少个地址？
(3) 每一个子网的地址块是什么？
(4) 每一个子网可分配给主机使用的最小地址和最大地址是什么？
- 4-34** IGP 和 EGP 这两类协议的主要区别是什么？
- 4-35** 试简述 RIP, OSPF 和 BGP 路由选择协议的主要特点。
- 4-36** RIP 使用 UDP, OSPF 使用 IP, 而 BGP 使用 TCP。这样做有何优点？为什么 RIP 周期性地和邻站交换路由信息而 BGP 却不这样做？
- 4-37** 假定网络中的路由器 B 的路由表有如下的项目（这三列分别表示“目的网络”“距离”和“下一跳路由器”）：

N ₁	7	A
N ₂	2	C
N ₆	8	F
N ₈	4	E
N ₉	4	F

现在 B 收到从 C 发来的路由信息（这两列分别表示“目的网络”和“距离”）：

N ₂	4
N ₃	8
N ₆	4
N ₈	3
N ₉	5

试求出路由器 B 更新后的路由表（详细说明每一个步骤）。

- 4-38** 网络如图 4-79 所示。假定 AS₁ 和 AS₄ 运行程序 RIP，AS₂ 和 AS₃ 运行程序 OSPF。AS 之间运行程序 eBGP 和 iBGP。目前先假定在 AS₂ 和 AS₄ 之间没有物理连接（图中的虚线表示这个假定）。

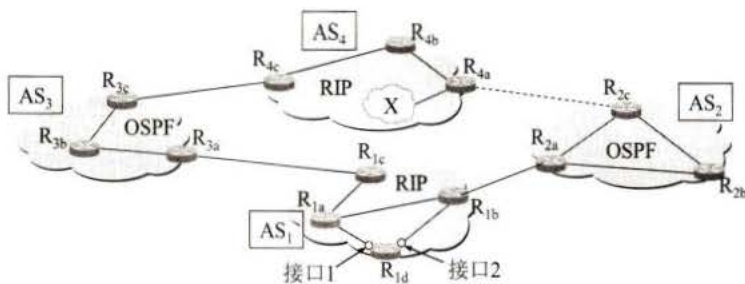


图 4-79 习题 4-38 的图

- (1) 路由器 R_{3c} 使用哪一个协议知道前缀 X（X 在 AS₄ 中）？
- (2) 路由器 R_{3a} 使用哪一个协议知道前缀 X？
- (3) 路由器 R_{1c} 使用哪一个协议知道前缀 X？
- (4) 路由器 R_{1d} 使用哪一个协议知道前缀 X？

- 4-39** 网络同上题。路由器 R_{1d} 知道前缀 X，并将前缀 X 写入转发表。

- (1) 试问路由器 R_{1d} 应当从接口 1 还是接口 2 转发分组呢？请简述理由。
- (2) 现假定 AS₂ 和 AS₄ 之间有物理连接，即图中的虚线变成了实线。假定路由器 R_{1d} 知道到达前缀 X 可以经过 AS₂，但也可以经过 AS₃。试问路由器 R_{1d} 应当从接口 1 还是接口 2 转发分组呢？请简述理由。
- (3) 现假定有另一个 AS₅ 处在 AS₂ 和 AS₄ 之间（图中的虚线之间未画出 AS₅）。假定路由器 R_{1d} 知道到达前缀 X 可以经过路由 [AS₂ AS₅ AS₄]，但也可以经过路由 [AS₃ AS₄]。试问路由器 R_{1d} 应当从接口 1 还是接口 2 转发分组呢？请简述理由。

- 4-40** IGMP 协议的要点是什么？隧道技术在多播中是怎样使用的？

- 4-41** 什么是 VPN？VPN 有什么特点和优缺点？VPN 有几种类别？

- 4-42** 什么是 NAT？NAPT 有哪些特点？NAT 的优点和缺点有哪些？

- 4-43** 试把下列 IPv4 地址从二进制记法转换为点分十进制记法。

- (1) 10000001 00001011 00001011 11101111
- (2) 11000001 10000011 00011011 11111111
- (3) 11100111 11011011 10001011 01101111
- (4) 11111001 10011011 11111011 00001111

- 4-44** 假设一段地址的首地址为 146.102.29.0，末地址为 146.102.32.255，求这个地址段的

地址数。

- 4-45 已知一/27 网络中有一个地址是 167.199.170.82，问这个网络的网掩码、网络前缀长度和网络后缀长度是多少？
- 4-46 已知条件同上题，试求这个地址块的地址数、首地址以及末地址各是多少？
- 4-47 某单位分配到一个地址块 14.24.74.0/24。该单位需要用到三个子网，对这三个子地址块的具体要求是：子网 N_1 需要 120 个地址，子网 N_2 需要 60 个地址，子网 N_3 需要 10 个地址。请给出地址块的分配方案。
- 4-48 如图 4-80 所示，网络 145.13.0.0/16 划分为四个子网 N_1 、 N_2 、 N_3 和 N_4 。这四个子网与路由器 R 连接的接口分别是 m0、m1、m2 和 m3。路由器 R 的第五个接口 m4 连接到互联网。

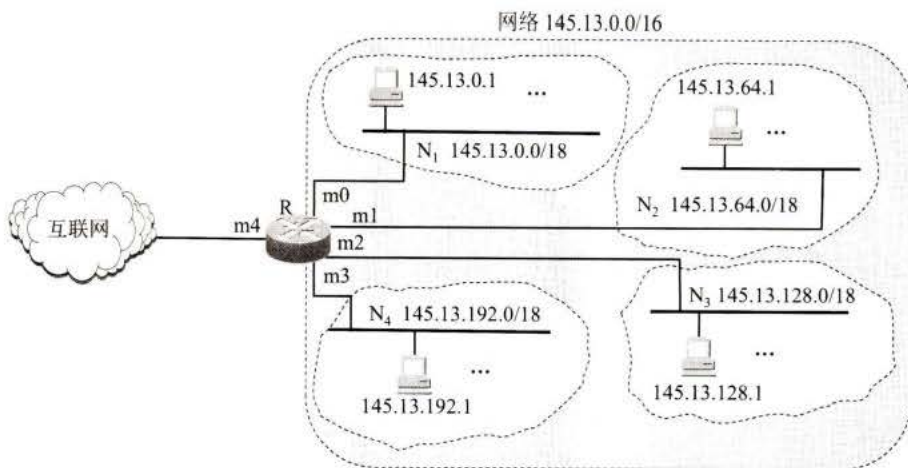


图 4-80 习题 4-48 的图

- (1) 试给出路由器 R 的路由表。
- (2) 路由器 R 收到一个分组，其目的地址是 145.13.160.78。试给出这个分组是怎样被转发的。
- 4-49 收到一个分组，其目的地址 $D = 11.1.2.5$ 。要查找的转发表中有这样三项：
- 路由 1 到达网络 11.0.0.0/8
- 路由 2 到达网络 11.1.0.0/16
- 路由 3 到达网络 11.1.2.0/24
- 试问在转发这个分组时应当选择哪一个路由？
- 4-50 同上题。假定路由 1 的目的网络 11.0.0.0/8 中有一台主机 H，其 IP 地址是 11.1.2.3。当我们发送一个分组给主机 H 时，根据最长前缀匹配准则，上面的这个转发表却把这个分组转发到路由 3 的目的网络 11.1.2.0/24。是最长前缀匹配准则有时会出错吗？
- 4-51 已知一 CIDR 地址块为 200.56.168.0/21。
- (1) 试用二进制形式表示这个地址块。
- (2) 这个 CIDR 地址块包括有多少个 C 类地址块？
- 4-52 建议的 IPv6 协议没有首部检验和。这样做的优缺点是什么？
- 4-53 在 IPv4 首部中有一个“协议”字段，但在 IPv6 的固定首部中却没有。这是为什么？

- 4-54 当使用 IPv6 时，协议 ARP 是否需要改变？如果需要改变，那么应当进行概念性的改变还是技术性的改变？
- 4-55 IPv6 只允许在源点进行分片。这样做有什么好处？
- 4-56 设每隔 1 微微秒就分配出 100 万个 IPv6 地址。试计算大约要用多少年才能将 IPv6 地址空间全部用光。可以和宇宙的年龄（大约有 100 亿年）进行比较。
- 4-57 试把以下的 IPv6 地址用零压缩方法写成简洁形式：
- (1) 0000:0000:0F53:6382:AB00:67DB:BB27:7332
 - (2) 0000:0000:0000:0000:0000:0000:004D:ABCD
 - (3) 0000:0000:0000:AF36:7328:0000:87AA:0398
 - (4) 2819:00AF:0000:0000:0000:0035:0CB2:B271
- 4-58 试把以下的零压缩的 IPv6 地址写成原来的形式：
- (1) 0::0
 - (2) 0:AA::0
 - (3) 0:1234::3
 - (4) 123::1:2
- 4-59 从 IPv4 过渡到 IPv6 的方法有哪些？
- 4-60 多协议标签交换 MPLS 的工作原理是怎样的？它有哪些主要的功能？
- 4-61 SDN 的广义转发与传统的基于终点的转发有何区别？
- 4-62 试举出 IP 数据报首部中能够在 OpenFlow 1.0 中匹配的三个字段。试举出在 OpenFlow 中不能匹配的三个 IP 数据报首部。
- 4-63 网络如图 4-81 所示。
- (1) 假定路由器 R_1 把所有发往网络前缀 128.1.2.16/29 的分组都从接口 4 转发出去。
 - (2) 假定路由器 R_1 要把 H_1 发往 128.1.2.16/29 的分组从接口 4 转发出去，而把 H_2 发往 128.1.2.16/29 的分组从接口 3 转发出去。
- 试问，在上述两种情况下，你都能够给出路由器 R_1 的转发表吗？转发表只需要给出发往 128.1.2.16/29 的分组应当从哪一个接口转发出去。

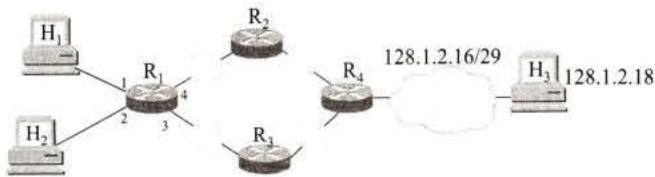


图 4-81 习题 4-63 的网络

- 4-64 已知一具有 4 个接口的路由器 R_1 的转发表如表 4-11 所示，转发表的每一行给出了目的地址的范围，以及对应的转发接口。

表 4-11 习题 4-64 中路由器 R_1 的转发表

目的地址范围	转发接口
最小地址 11100000 00000001 00000000 00000000	0
最大地址 11100000 00000001 11111111 11111111	

续表

目的地址范围	转发接口
最小地址 11100000 00000000 00000000 00000000 最大地址 11100000 00000000 11111111 11111111	1
最小地址 11100000 00000010 00000000 00000000 最大地址 11100001 11111111 11111111 11111111	2
其他	3

(1) 试把以上转发表改换为另一形式，其中的目的地址范围改为前缀匹配，而转发表由 4 行增加为 5 行。

(2) 若路由器收到一个分组，其目的地址是：

(a) 11100000 10000001 01010001 01010101

(b) 11100000 00000000 11010111 01111100

(c) 11100001 10010000 00010001 01110111

试给出每一种情况下分组应当通过的转发接口。

4-65 一路由器连接到三个子网，这三个子网共同的前缀是 225.2.17/24。假定子网 N_1 要有 62 台主机，子网 N_2 要有 105 台主机，而子网 N_3 要有 12 台主机。试分配这三个子网的前缀。

4-66 图 4-82 是一个 SDN OpenFlow 网络。假定：

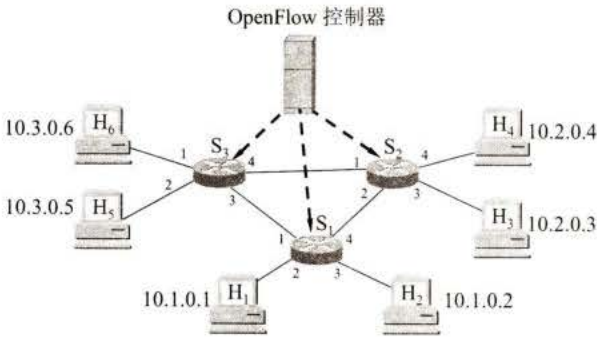


图 4-82 一个 SDN OpenFlow 网络

- 任何来自 H_5 或 H_6 并到达输入端口 1 的且发往 H_1 或 H_2 的分组，应通过端口 2 转发出去。
- 任何来自 H_1 或 H_2 并到达输入端口 2 的且发往 H_5 或 H_6 的分组，应通过端口 1 转发出去。
- 任何从端口 1 或 2 进入的且发往 H_3 或 H_4 的分组，应交付指明的主机；
- H_3 或 H_4 彼此可以互相发送分组。

试给出交换机 S_2 的流表项（即每一行的“匹配 + 动作”）。

4-67 SDN OpenFlow 网络同上题。从主机 H_3 或 H_4 发出并到达 S_2 交换机的分组，应遵循以下规则：

- 任何来自 H_3 并到达输入端口 2 的且发往 H_1 , H_2 , H_5 或 H_6 的分组，应顺时针转发出去。

- 任何来自 H_4 并到达输入端口 2 的且发往 H_1 , H_2 , H_5 或 H_6 的分组, 应逆时针转发出去。

试给出交换机 S_2 的流表项 (即每一行的“匹配 + 动作”)。

4-68 SDN OpenFlow 网络同上题。在交换机 S_1 和 S_3 有这样的规定: 从源地址 H_3 或 H_4 到来的分组, 将按照分组首部中的目的地址进行转发。试给出交换机 S_1 和 S_3 的流表。

4-69 SDN OpenFlow 网络同上题。假定我们把交换机 S_2 作为防火墙。防火墙的行为有以下两种:

(1) 对于目的地址为 H_3 和 H_4 的分组, 仅可转发从 H_1 和 H_6 发出的分组, 也就是说, 从 H_1 和 H_5 发出的分组应当被阻挡。

(2) 仅对于目的地址为 H_3 的分组才交付, 也就是说, 所有发往 H_4 的分组均被阻挡。

试分别对上述的每一种情况给出交换机 S_2 的流表。对于发往其他路由器的分组可不用管。

第5章 运输层

本章先概括介绍运输层协议的特点、进程之间的通信和端口等重要概念，然后讲述比较简单的 UDP 协议。其余的篇幅都是讨论较为复杂但非常重要的 TCP 协议^①和可靠传输的工作原理，包括停止等待协议和 ARQ 协议。在详细讲述 TCP 报文段的首部格式之后，讨论 TCP 的三个重要问题：滑动窗口、流量控制和拥塞控制机制。最后，介绍 TCP 的连接管理。

运输层是整个网络体系结构中的关键层次之一。一定要弄清以下一些重要概念：

- (1) 运输层为相互通信的应用进程提供逻辑通信。
- (2) 端口和套接字的意义。
- (3) 无连接的 UDP 的特点。
- (4) 面向连接的 TCP 的特点。
- (5) 在不可靠的网络上实现可靠传输的工作原理，停止等待协议和 ARQ 协议。
- (6) TCP 的滑动窗口、流量控制、拥塞控制和连接管理。

5.1 运输层协议概述

5.1.1 进程之间的通信

从通信和信息处理的角度看，**运输层向它上面的应用层提供通信服务**，它属于面向通信部分的最高层，同时也是用户功能中的最低层。当网络边缘部分的两台主机使用网络核心部分的功能进行端到端的通信时，都要使用协议栈中的运输层，而网络核心部分中的路由器在转发分组时只用到下三层的功能。

下面通过图 5-1 的示意图来说明运输层的作用。设局域网 LAN₁ 上的主机 A 和局域网 LAN₂ 上的主机 B 通过互连的广域网 WAN 进行通信。我们知道，IP 协议能够把源主机 A 发送出的分组，按照首部中的目的地址，送交到目的主机 B，那么，为什么还需要运输层呢？

从 IP 层来说，通信的两端是两台主机。IP 数据报的首部明确地标志了这两台主机的 IP 地址。但“两台主机之间的通信”这种说法还不够明确。真正进行通信的实体是在主机中的哪个构件呢？是主机中的应用进程，是一台主机中的**应用进程**和另一台主机中的**应用进程**在交换数据（即通信）。因此严格地讲，两台主机进行通信就是两台主机中的**应用进程互相通信**。IP 协议虽然能把分组送到目的主机，但是这个分组还停留在主机的网络层而没有交付主机中的应用进程。**通信的两端应当是两个主机中的应用进程**。也就是说，**端到端的通信**是应用进程之间的通信。在一台主机中经常有多个应用进程同时分别和另一台主机中的多个应用进程通信。例如，某用户在使用浏览器查找某网站的信息时，其主机的应用层运行浏览器客户进程。如果在浏览网页的同时，还要用电子邮件给网站发送反馈意见，那么主机的应用

^① 注：运输层最近又增加了第三种协议，即**流控制传输协议 SCTP** (Stream Control Transmission Protocol) [RFC 4960，建议标准]，它具有 TCP 和 UDP 协议的共同优点，可支持一些新的应用，如 IP 电话。限于篇幅，这里不再介绍。

层就还要运行电子邮件的客户进程。在图 5-1 中，主机 A 的应用进程 AP_1 和主机 B 的应用进程 AP_3 通信，而与此同时，应用进程 AP_2 也和对方的应用进程 AP_4 通信。这表明运输层有一个很重要的功能——复用(multiplexing)和分用(demultiplexing)。这里的“复用”是指在发送方不同的应用进程都可以使用同一个运输层协议传送数据（当然需要加上适当的首部），而“分用”是指接收方的运输层在剥去报文的首部后能够把这些数据正确交付目的应用进程^①。图 5-1 中两个运输层之间有一个深色双向粗箭头，写明“运输层提供应用进程间的逻辑通信”。“逻辑通信”的意思是：从应用层来看，只要把应用层报文交给下面的运输层，运输层就可以把这报文传送到对方的运输层（哪怕双方相距很远，例如几千公里），好像这种通信就是沿水平方向直接传送数据。但事实上这两个运输层之间并没有一条水平方向的物理连接。数据的传送是沿着图中的虚线方向（经过多个层次）传送的。“逻辑通信”的意思是“好像是这样通信，但事实上并非真的这样通信”。

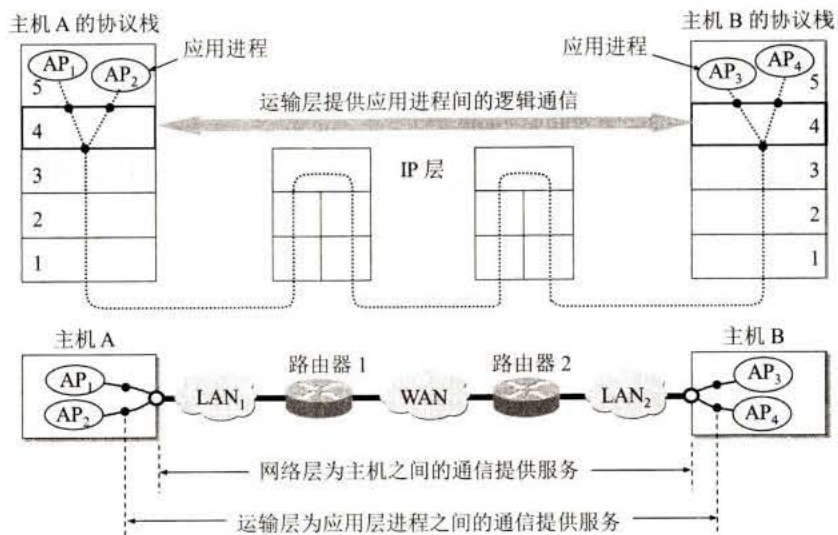


图 5-1 运输层为相互通信的应用进程提供了逻辑通信

从这里可以看出网络层和运输层有明显的区别。网络层为主机之间的通信提供服务，而运输层则在网络层的基础上，为应用进程之间的通信提供服务。然而正如后面还要讨论的，运输层还具有网络层无法代替的许多其他重要功能。

运输层还要对收到的报文进行差错检测。大家应当还记得，在网络层，IP 数据报首部中的检验和字段，只检验首部是否出现差错而不检查数据部分。

根据应用程序的不同需求，运输层需要有两种不同的运输协议，即面向连接的 TCP 和无连接的 UDP，这两种协议就是本章要讨论的主要内容。

我们还应指出，运输层向高层用户屏蔽了下面网络核心的细节（如网络拓扑、所采用的路由选择协议等），它使应用进程看见的就是好像在两个运输层实体之间有一条端到端的逻辑通信信道，但这条逻辑通信信道对上层的表现却因运输层使用的不同协议而有很大的差别。当运输层采用面向连接的 TCP 协议时，尽管下面的网络是不可靠的（只提供尽最大努

^① 注：IP 层也有复用和分用的功能。即在发送方使用不同协议的数据都可以封装成 IP 数据报发送出去，而在接收方的 IP 层根据 IP 首部中的协议字段进行分用，把剥去首部后的数据交付应当接收这些数据的协议。

力服务), 但这种逻辑通信信道就相当于一**条全双工的可靠信道**。但当运输层采用无连接的 **UDP 协议**时, 这种逻辑通信信道仍然是一条**不可靠信道**。

5.1.2 运输层的两个主要协议

TCP/IP 运输层的两个主要协议都是互联网的正式标准, 即:

- (1) 用户数据报协议 UDP (User Datagram Protocol) [RFC 768, STD6]
- (2) 传输控制协议 TCP (Transmission Control Protocol) [RFC 793, STD7]

图 5-2 给出了这两种协议在协议栈中的位置。



图 5-2 TCP/IP 体系中的运输层协议

按照 OSI 的术语, 两个对等运输实体在通信时传送的数据单位叫作**运输协议数据单元 TPDU (Transport Protocol Data Unit)**。但在 TCP/IP 体系中, 则根据所使用的协议是 TCP 或 UDP, 分别称之为 **TCP 报文段(segment)** 或 **UDP 用户数据报**。

UDP 在传送数据之前**不需要先建立连接**。远地主机的运输层在收到 UDP 报文后, 不需要给出任何确认。虽然 UDP 不提供可靠交付, 但由于 UDP 非常简单, 在某些情况下 UDP 是一种最有效的工作方式。

TCP 则**提供面向连接的服务**。在传送数据之前必须先建立连接, 数据传送结束后要释放连接。TCP 不提供广播或多播服务。由于 TCP 要提供可靠的、面向连接的运输服务, 因此不可避免地增加了许多的开销, 如确认、流量控制、计时器以及连接管理等。这不仅使协议数据单元的首部增大很多, 还要占用许多的处理机资源。

表 5-1 给出了一些应用和应用层协议主要使用的运输层协议 (UDP 或 TCP)。

表 5-1 使用 UDP 或 TCP 协议的各种应用和应用层协议

应用	应用层协议	运输层协议
名字转换	DNS (域名系统)	UDP
文件传送	TFTP (简单文件传送协议)	UDP
路由选择协议	RIP (路由信息协议)	UDP
IP 地址配置	DHCP (动态主机配置协议)	UDP
网络管理	SNMP (简单网络管理协议)	UDP
远程文件服务器	NFS (网络文件系统)	UDP
IP 电话	专用协议	UDP
流式多媒体通信	专用协议	UDP
多播	IGMP (网际组管理协议)	UDP
电子邮件	SMTP (简单邮件传送协议)	TCP
远程终端接入	TELNET (远程终端协议)	TCP
万维网	HTTP (超文本传送协议)	TCP
文件传送	FTP (文件传送协议)	TCP

5.1.3 运输层的端口

前面已经提到过运输层的复用和分用功能。其实在日常生活中也有很多复用和分用的例子。假定一个机构的所有部门向外单位发出的公文都由收发室负责寄出，这相当于各部门都“复用”这个收发室。当收发室收到从外单位寄来的公文时，则要完成“分用”功能，即按照信封上写明的本机构的部门地址把公文正确进行交付。

运输层的复用和分用功能也是类似的。应用层所有的应用进程都可以通过运输层再传送到 IP 层（网络层），这就是**复用**。运输层从 IP 层收到发送给各应用进程的数据后，必须分别交付指明的各应用进程，这就是**分用**。显然，给应用层的每个应用进程赋予一个非常明确的标志是至关重要的。

我们知道，在单个计算机中的进程是用进程标识符（一个不大的整数）来标志的。但是在互联网环境下，用计算机操作系统所指派的这种进程标识符来标志运行在应用层的各种应用进程则是不行的。这是因为在互联网上使用的计算机的操作系统种类很多，而不同的操作系统又使用不同格式的进程标识符。为了使运行不同操作系统的计算机的应用进程能够互相通信，就必须用统一的方法（而这种方法必须与特定操作系统无关）对 TCP/IP 体系的应用进程进行标志。

但是，把一个特定机器上运行的特定进程，指明为互联网上通信的最后终点是不可行的。这是因为进程的创建和撤销都是动态的，通信的一方几乎无法知道和识别对方机器上的进程。另外，我们往往需要利用目的主机提供的功能来识别终点，而不需要知道具体实现这个功能的进程是哪一个（例如，要和互联网上的某个邮件服务器联系，但并不一定要知道这个服务器功能是由目的主机上的哪个进程实现的）。

解决这个问题的方法很巧妙，就是在应用层和运输层之间的界面上，设置一个特殊的抽象的“门”。应用层中的应用进程要通过运输层发送到互联网，必须要通过这个门。而别的主机上的应用进程要寻找本主机中的某个应用进程，也必须通过这个门。这样，我们就可以把应用层和运输层的界面上这些“门”，设为通信的**抽象终点**。这些抽象终点的正式名称就是**协议端口(protocol port)**，一般就简称为**端口(port)**。每一个端口用一个称为**端口号(port number)**的正整数来标志。主机的操作系统提供了接口机制，使得进程能够通过这种机制找到所要找的端口。

请注意，这种在协议栈层间的抽象的协议端口是软件端口，和路由器或交换机上的硬件端口是完全不同的概念。硬件端口是不同硬件设备进行交互的接口，而软件端口是应用层的各种协议进程与运输实体进行层间交互的地点。不同的系统具体实现端口的方法可以是不一样的（取决于系统使用的操作系统）。

当应用层要发送数据时，应用进程就把数据发送到适当的端口，然后运输层从该端口读取数据，进行后续的处理（把数据发送到目的主机）。当运输层收到对方主机发来的数据时，就把数据发送到适当的端口，然后应用进程就从该端口读取这些数据。显然，端口必须有一定容量的缓存来暂时存放数据。

在后面将讲到的 UDP 和 TCP 的首部格式中（图 5-5 和图 5-14），都有**源端口**和**目的端口**这两个重要字段。这两个端口就是运输层和应用层进行交互的地点。

TCP/IP 的运输层用一个 16 位端口号来标志一个端口。但请注意，**端口号只具有本地意义**，它只是为了标志本计算机应用层中的各个进程在和运输层交互时的层间接口。在互联网不同计算机中，相同的端口号是没有关联的。16 位的端口号可允许有 65535 个不同的端口

号，这个数目对一个计算机来说是足够用的。

由此可见，两个计算机中的进程要互相通信，不仅必须知道对方的 IP 地址（为了找到对方的计算机），而且要知道对方的端口号（为了找到对方计算机中的应用进程）。这和我们寄信的过程类似。当我们要给某人写信时，就必须在信封上写明他的通信地址（这是为了找到他的住所，相当于 IP 地址），并且还要写上收件人的姓名（这是因为在同一住所中可能有好几个人，这相当于端口号）。在信封上还应写明自己的地址和姓名。当收信人回信时，很容易在信封上找到发信人的地址和姓名。互联网上的计算机通信是采用客户-服务器方式。客户在发起通信请求时，必须先知道对方服务器的 IP 地址（用来找到目的主机）和端口号（用来找到目的进程）。因此运输层的端口号分为下面的两大类。

(1) 服务器端使用的端口号 这里又分为两类，最重要的一类叫作熟知端口号(well-known port number)或全球通用端口号，数值为 0~1023。这些熟知端口号最初公布在文档中[RFC 1700, STD2]，但后来因为互联网发展太快，这种标准文档无法随时更新，因此在 RFC 3232 中就把 RFC 1700 列为陈旧的，而当前最新的熟知端口号可在网址 www.iana.org 上查到。IANA 把这些熟知端口号指派给了 TCP/IP 最重要的一些应用程序，让所有的用户都知道。当一种新的应用程序出现后，IANA 必须为它指派一个熟知端口，否则互联网上的其他应用进程就无法和它进行通信。和电话通信对比，熟知端口号相当于所有人都应知晓的重要电话号码，如报警电话 110，急救电话 120 等。表 5-2 给出了一些常用的熟知端口号。

表 5-2 常用的熟知端口号

应用程序	FTP	TELNET	SMTP	DNS	TFTP	HTTP	SNMP	SNMP (trap)	HTTPS
熟知端口号	21	23	25	53	69	80	161	162	443

另一类叫作登记端口号，数值为 1024~49151。这类端口号是为没有熟知端口号的应用程序使用的。要使用这类端口号必须在 IANA 按照规定的手续登记，以防止重复。

(2) 客户端使用的端口号 数值为 49152~65535。由于这类端口号仅在客户进程运行时才动态选择，因此又叫作短暂端口号^①。这类端口号就是临时端口号，留给客户进程选择临时使用。当服务器进程收到客户进程的报文时，就知道了客户进程所使用的端口号，因而可以把数据发送给客户进程。通信结束后，刚才已使用过的客户端口号就被系统收回，以便给其他客户进程使用。

下面将分别讨论 UDP 和 TCP。UDP 比较简单，本章主要讨论 TCP。

5.2 用户数据报协议 UDP

5.2.1 UDP 概述

用户数据报协议 UDP 只在 IP 的数据报服务之上增加了很少一点的功能，这就是复用和分用的功能以及差错检测的功能。UDP 的主要特点是：



^① 注：短暂端口(ephemeral port)[STEV94, p.13]表示这种端口的存在时间是短期的。客户进程并不在意操作系统给它分配的是哪一个端口号，因为客户进程之所以必须有一个端口号（在本地主机中必须是唯一的），是为了让运输层的实体能够找到自己。这和熟知端口不同。服务器机器一接通电源，服务器程序就运行起来。为了让互联网上所有的客户程序都能找到服务器程序，服务器程序所使用的端口（即熟知端口）就必须是固定的，并且是众所周知的。

(1) UDP 是无连接的，即发送数据之前不需要建立连接（当然，发送数据结束时也没有连接可释放），因此减少了开销和发送数据之前的时延。

(2) UDP 使用尽最大努力交付，即不保证可靠交付，因此主机不需要维持复杂的连接状态表（这里面有许多参数）。

(3) UDP 是面向报文的。发送方的 UDP 对应用程序交下来的报文，在添加首部后就向下交付 IP 层。UDP 对应用层交下来的报文，既合并，也不拆分，而是保留这些报文的边界。这就是说，应用层交给 UDP 多长的报文，UDP 就照样发送，即一次发送一个报文，如图 5-3 所示。在接收方的 UDP，对 IP 层交上来的 UDP 用户数据报，在去除首部后就原封不动地交付上层的应用进程。也就是说，UDP 一次交付一个完整的报文。因此，应用程序必须选择合适大小的报文。若报文太长，UDP 把它交给 IP 层后，IP 层在传送时可能要进行分片，这会降低 IP 层的效率。反之，若报文太短，UDP 把它交给 IP 层后，会使 IP 数据报的首部的相对长度太大，这也降低了 IP 层的效率。

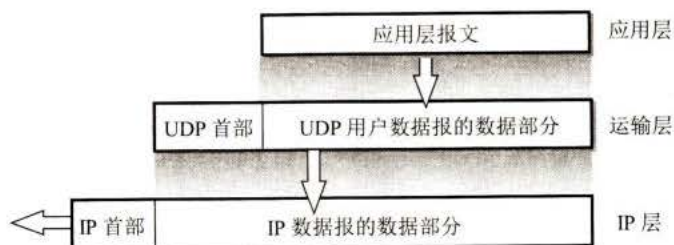


图 5-3 UDP 是面向报文的

(4) UDP 没有拥塞控制，因此网络出现的拥塞不会使源主机的发送速率降低。这对某些实时应用是很重要的。很多的实时应用（如 IP 电话、实时视频会议等）要求源主机以恒定的速率发送数据，并且允许在网络发生拥塞时丢失一些数据，但却不允许数据有太大的时延。UDP 正好适合这种要求。

(5) UDP 支持一对一、一对多、多对一和多对多的交互通信。

(6) UDP 的首部开销小，只有 8 个字节，比 TCP 的 20 个字节的首部要短。

下面举例说明 UDP 的通信和端口号的关系（如图 5-4 所示）。主机 H_1 中有三个应用进程分别要和主机 H_2 中的两个应用进程进行通信。通信双方的关系是： $P_1 \rightarrow P_4$ ， $P_2 \rightarrow P_4$ ， $P_3 \rightarrow P_5$ 。主机 H_1 的操作系统为这三个进程分别指派了端口，其端口号分别为 a 、 b 和 c 。图中位于应用层和运输层之间的小方框代表端口。在端口小方框中间还画有队列，表示端口具有缓存的功能，可以把收到的数据暂时存储一下。有时也可以把队列画成双向的，即分别表示存放来自应用层或运输层的数据。现在假定主机 H_1 中的进程已经知道了对方进程 P_4 和 P_5 的端口号分别为 x 和 y ，于是在主机 H_1 的运输层就可以组装成需要发送的 UDP 用户数据报，其中最重要的地址信息（源端口，目的端口）分别是 (a, x) 、 (b, x) 和 (c, y) 。在图 5-4 中把运输层以下的都省略了。因此，进程之间的通信现在可以看成是两个端口之间的通信。

图中在两个运输层之间有一条虚线，表示在两个运输层之间可以进行通信，而不是一条连接。但这种通信是不可靠的通信，即所发送的报文在传输过程中有可能丢失，同时也不保证报文都能按照发送的先后顺序到达终点。这正是 UDP 通信的特点：简单方便，但不可靠。如果想要得到可靠的运输层通信，那就要使用后面要介绍的 TCP 进行通信。请注意，在两个运输层的 UDP 之间并没有建立连接。

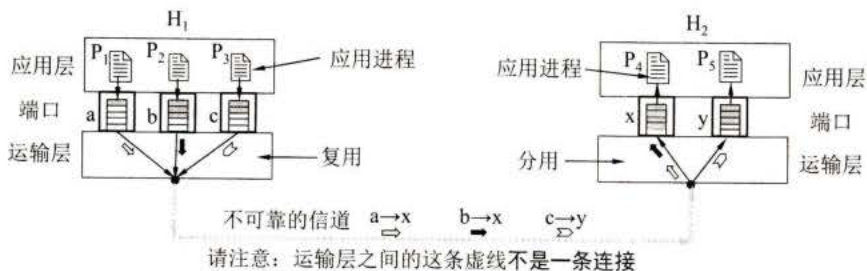


图 5-4 UDP 的通信和端口号的关系

图 5-4 的例子画出了多对一的通信 ($a \rightarrow x$, $b \rightarrow x$)。如果改成 $a \rightarrow x$, $a \rightarrow y$, 则是一对多的情况了。

主机 H_1 中的 3 个应用进程, 把用户数据通过各自的端口传送到运输层后, 就共用一个网络层协议, 把收到的 UDP 用户数据报组装成不同的 IP 数据报, 发送到互联网。这就是 UDP 的复用功能。主机 H_2 的网络层收到 3 个 IP 数据报后, 提取出数据部分 (即 UDP 用户数据报), 然后根据其首部中的目的端口号, 分别传送到相应的端口, 以便上层的应用进程到端口读取数据。这就是 UDP 的分用功能。

虽然某些实时应用需要使用没有拥塞控制的 UDP, 但当很多的源主机同时都向网络发送高速率的实时视频流时, 网络就有可能发生拥塞, 结果大家都无法正常接收。因此, 不使用拥塞控制功能的 UDP 有可能会引起网络产生严重的拥塞问题。

还有一些使用 UDP 的实时应用, 需要对 UDP 的不可靠的传输进行适当的改进, 以减少数据的丢失。在这种情况下, 应用进程本身可以在不影响应用的实时性的前提下, 增加一些提高可靠性的措施, 如采用前向纠错或重传已丢失的报文。

5.2.2 UDP 的首部格式

用户数据报 UDP 有两个字段: 数据字段和首部字段。首部字段很简单, 只有 8 个字节 (如图 5-5 所示), 由 4 个字段组成, 每个字段的长度都是 2 字节。各字段意义如下:

- (1) 源端口 源端口号。在需要对方回信时选用。不需要时可用全 0。
- (2) 目的端口 目的端口号。这在终点交付报文时必须使用。
- (3) 长度 UDP 用户数据报的长度, 其最小值是 8 (仅有首部)。
- (4) 检验和 检测 UDP 用户数据报在传输中是否有错。有错就丢弃。

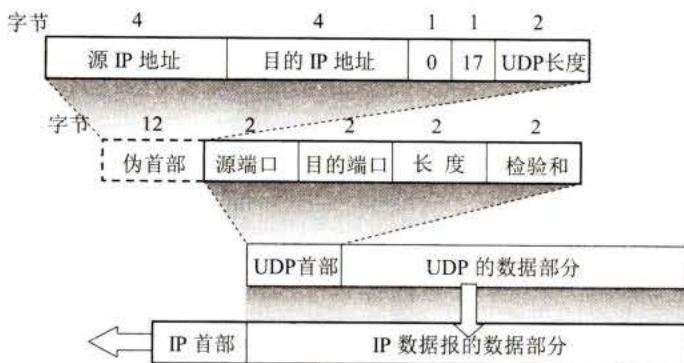


图 5-5 UDP 用户数据报的首部和伪首部

如果接收方 UDP 发现收到的报文中的目的端口号不正确（即不存在对应于该端口号的应用进程），就丢弃该报文，并由网际控制报文协议 ICMP 发送“端口不可达”差错报文给发送方。我们在上一章 4.4.2 节“ICMP 的应用举例”讨论 traceroute 时，就是让发送的 UDP 用户数据报故意使用一个非法的 UDP 端口，结果 ICMP 就返回“端口不可达”差错报文，因而达到了测试的目的。

UDP 用户数据报首部中检验和的计算方法有些特殊。在计算检验和时，要在 UDP 用户数据报之前增加 12 个字节的伪首部。所谓“伪首部”是因为这种伪首部并不是 UDP 用户数据报真正的首部。只是在计算检验和时，临时添加在 UDP 用户数据报前面，得到一个临时的 UDP 用户数据报。检验和就是按照这个临时的 UDP 用户数据报来计算的。伪首部既不向下传送也不向上递交，而仅仅是为了计算检验和。图 5-5 的最上面给出了伪首部各字段的内容。

UDP 计算检验和的方法和计算 IP 数据报首部检验和的方法相似。但不同的是：IP 数据报的检验和只检验 IP 数据报的首部，但 UDP 的检验和是把首部和数据部分一起都检验。在发送方，首先是先把全零放入检验和字段。再把伪首部以及 UDP 用户数据报看成是由许多 16 位的字串接起来的。若 UDP 用户数据报的数据部分不是偶数个字节，则要填入一个全零字节（但此字节不发送）。然后按二进制反码计算出这些 16 位字的和。将此和的二进制反码写入检验和字段后，就发送这样的 UDP 用户数据报。在接收方，把收到的 UDP 用户数据报连同伪首部（以及可能的填充全零字节）一起，按二进制反码求这些 16 位字的和。当无差错时其结果应为全 1。否则就表明有差错出现，接收方就应丢弃这个 UDP 用户数据报（也可以上交给应用层，但附上出现了差错的警告）。图 5-6 给出了一个计算 UDP 检验和的例子。这里假定用户数据报的长度是 15 字节，因此要添加一个全 0 的字节。读者可以自己检验一下在接收端是怎样对检验和进行检验的。不难看出，这种简单的差错检验方法的检错能力并不强，但它的好处是简单，处理起来较快。



图 5-6 计算 UDP 检验和的例子

如图 5-6 所示，伪首部的第 3 字段是全零；第 4 字段是 IP 首部中的协议字段的值。以前曾讲过，对于 UDP，此协议字段值为 17；第 5 字段是 UDP 用户数据报的长度。因此，这样的检验和，既检查了 UDP 用户数据报的源端口号和目的端口号以及 UDP 用户数据报的数据部分，又检查了 IP 数据报的源 IP 地址和目的地址。

5.3 传输控制协议 TCP 概述

扫一扫



视频讲解

由于 TCP 协议比较复杂,因此本节先对 TCP 协议进行一般的介绍,然后再逐步深入讨论 TCP 的可靠传输、流量控制和拥塞控制等问题。

5.3.1 TCP 最主要的特点

TCP 是 TCP/IP 体系中非常复杂的一个协议。下面介绍 TCP 最主要的特点。

(1) TCP 是面向连接的运输层协议。这就是说,应用程序在使用 TCP 协议之前,必须先建立 TCP 连接。在传送数据完毕后,必须释放已经建立的 TCP 连接。也就是说,应用进程之间的通信好像在“打电话”:通话前要先拨号建立连接,通话结束后要挂机释放连接。

(2) 每一条 TCP 连接只能有两个端点(endpoint),每一条 TCP 连接只能是点对点的(一对一)。这个问题后面还要进一步讨论。

(3) TCP 提供可靠交付的服务。通过 TCP 连接传送的数据,无差错、不丢失、不重复,并且按序到达。

(4) TCP 提供全双工通信。TCP 允许通信双方的应用进程在任何时候都能发送数据。TCP 连接的两端都设有发送缓存和接收缓存,用来临时存放双向通信的数据。在发送时,应用程序在把数据传送给 TCP 的缓存后,就可以做自己的事,而 TCP 在合适的时候把数据发送出去。在接收时, TCP 把收到的数据放入缓存,上层的应用进程在合适的时候读取缓存中的数据。

(5) 面向字节流。TCP 中的“流”(stream)指的是流入到进程或从进程流出的字节序列。“面向字节流”的含义是:虽然应用程序和 TCP 的交互是一次一个数据块(大小不等),但 TCP 把应用程序交下来的数据仅仅看成是一连串的无结构的字节流。TCP 并不知道所传送的字节流的含义。TCP 不保证接收方应用程序所收到的数据块和发送方应用程序所发出的数据块具有对应大小的关系(例如,发送方应用程序交给发送方的 TCP 共 10 个数据块,但接收方的 TCP 可能只用了 4 个数据块就把收到的字节流交付上层的应用程序)。但接收方应用程序收到的字节流必须和发送方应用程序发出的字节流完全一样。当然,接收方的应用程序必须有能力识别收到的字节流,把它还原成有意义的应用层数据。图 5-7 是上述概念的示意图。

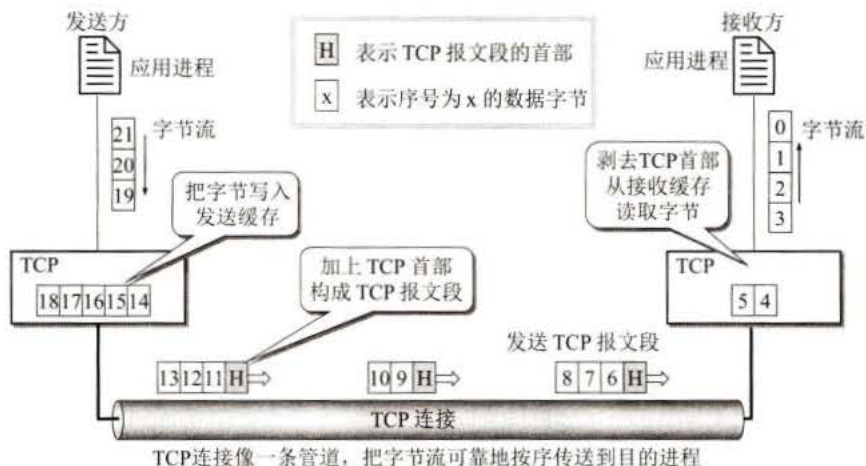


图 5-7 TCP 面向字节流的概念

为了突出示意图的要点，我们只画出了一个方向的数据流。但请注意，在实际的网络中，一个 TCP 报文段包含上千个字节是很常见的，而图中的各部分都只画出了几个字节，这仅仅是为了更方便地说明“面向字节流”的概念。另一点很重要：图 5-7 中的 TCP 连接是一条虚连接（也就是逻辑连接），而不是一条真正的物理连接。TCP 报文段先要传送到 IP 层，加上 IP 首部后，再传送到数据链路层；再加上数据链路层的首部和尾部后，才离开主机发送到物理链路。

从图 5-7 可看出，TCP 和 UDP 在发送报文时所采用的方式完全不同。TCP 并不关心应用进程一次把多长的报文发送到 TCP 的缓存中，而是根据对方给出的窗口值和当前网络拥塞的程度（后面还将深入讨论），来决定一个报文段应包含多少个字节（UDP 发送的报文长度是应用进程给出的）。如果应用进程传送到 TCP 缓存的数据块太长，TCP 就可以把它划分为短一些的数据块再传送。如果应用进程一次只发来一个字节，TCP 也可以等待积累足够多的字节后再构成报文段发送出去。关于 TCP 报文段的长度问题，在后面还要进行讨论。

5.3.2 TCP 的连接

TCP 把连接作为最基本的抽象。TCP 的许多特性都与 TCP 是面向连接的这个基本特性有关。因此我们对 TCP 连接需要有更清楚的了解。

前面已经讲过，每一条 TCP 连接有两个端点。那么，TCP 连接的端点是什么呢？不是主机，不是主机的 IP 地址，不是应用进程，也不是运输层的协议端口。TCP 连接的端点叫作套接字(socket)或插口。根据 RFC 793 的定义：端口号拼接到(concatenated with) IP 地址即构成了套接字。因此，套接字的表示方法是在点分十进制的 IP 地址后面写上端口号，中间用冒号或逗号隔开。例如，若 IP 地址是 192.3.4.5 而端口号是 80，那么得到的套接字就是 (192.3.4.5: 80)。总之，我们有

$$\text{套接字 socket} = (\text{IP 地址: 端口号}) \quad (5-1)$$

每一条 TCP 连接唯一地被通信两端的两个端点（即套接字对 socket pair）所确定。即：

$$\text{TCP 连接} ::= \{\text{socket}_1, \text{socket}_2\} = \{(\text{IP}_1: \text{port}_1), (\text{IP}_2: \text{port}_2)\} \quad (5-2)$$

这里 IP_1 和 IP_2 分别是两个端点主机的 IP 地址，而 port_1 和 port_2 分别是两个端点主机中的端口号。因此，TCP 连接就是两个套接字 socket_1 和 socket_2 之间的连接。套接字 socket 是个很抽象的概念，在下一章的 6.8 节还要对套接字进行更多的介绍。

总之，TCP 连接就是由协议软件所提供的一种抽象。虽然有时为了方便，我们也可以说，在一个应用进程和另一个应用进程之间建立了一条 TCP 连接，但一定要记住：**TCP 连接的端点是个很抽象的套接字，即（IP 地址：端口号）**。也还应记住：同一个 IP 地址可以有多个不同的 TCP 连接，而同一个端口号也可以出现在多个不同的 TCP 连接中。

本来 socket 的意思就是插座（或插口）。选用 socket 这个名词是相当准确的。其实一条 TCP 连接就像一条电缆线，其两端都各带有一个插头。把每一端的插头插入位于主机的应用层和运输层之间的插座(socket)后，两个主机之间的进程就可以通过这条电缆线进行可靠通信了。但插座这个名词很容易让人想起来是个硬件，而 socket 是个软件名词，这样“套接字”就成为 socket 的标准译名了。

请注意，socket 这个名词有时容易使人把一些概念弄混淆，因为随着互联网的不断发展

以及网络技术的进步，同一个名词 **socket** 却可表示多种不同的意思。例如：

(1) 允许应用程序访问连网协议的应用编程接口 **API** (Application Programming Interface)，即运输层和应用层之间的一种接口，称为 **socket API**，并简称为 **socket**。

(2) 在 **socket API** 中使用的一个函数名也叫作 **socket**。

(3) 调用 **socket** 函数的端点称为 **socket**，如“创建一个数据报 **socket**”。

(4) 调用 **socket** 函数时，其返回值称为 **socket** 描述符，可简称为 **socket**。

(5) 在操作系统内核中连网协议的 **Berkeley** 实现，称为 **socket 实现**。

上面的这些 **socket** 的意思都和本章所引用的 **RFC 793** 定义的 **socket**（指端口号拼接到 **IP** 地址）不同。请读者加以注意。

5.4 可靠传输的工作原理

我们知道，**TCP** 发送的报文段是交给 **IP** 层传送的。但 **IP** 层只能提供尽最大努力服务，也就是说，**TCP** 下面的网络所提供的是不可靠的传输。因此，**TCP** 必须采用适当的措施才能使得两个运输层之间的通信变得可靠。

理想的传输条件有以下两个特点：

(1) 传输信道不产生差错。

(2) 不管发送方以多快的速度发送数据，接收方总是来得及处理收到的数据。

在这样的理想传输条件下，不需要采取任何措施就能够实现可靠传输。

然而实际的网络都不具备以上两个理想条件。但我们可以使用一些可靠传输协议，当出现差错时让发送方重传出现差错的数据，同时在接收方来不及处理收到的数据时，及时告诉发送方适当降低发送数据的速率。这样一来，本来不可靠的传输信道就能够实现可靠传输了。下面从最简单的停止等待协议^①讲起。

5.4.1 停止等待协议

全双工通信的双方既是发送方也是接收方。下面为了讨论问题的方便，我们仅考虑 **A** 发送数据而 **B** 接收数据并发送确认。因此 **A** 叫作发送方，而 **B** 叫作接收方。因为这里是讨论可靠传输的原理，因此把传送的数据单元都称为分组，而并不考虑数据是在哪一个层次上传送的^②。“停止等待”就是每发送完一个分组就停止发送，等待对方的确认。在收到确认后再次发送下一个分组。

1. 无差错情况

停止等待协议可用图 5-8 来说明。图 5-8(a)是最简单的无差错情况。**A** 发送分组 M_1 ，发完就暂停发送，等待 **B** 的确认。**B** 收到了 M_1 就向 **A** 发送确认。**A** 在收到了对 M_1 的确认后，

① 注：在计算机网络发展初期，通信链路不太可靠，因此在链路层传送数据时都要采用可靠的通信协议。其中最简单的协议就是这种“停止等待协议”。在运输层并不使用这种协议，这里只是为了引出可靠传输的问题才从最简单的概念讲起。在运输层使用的可靠传输协议要复杂得多（见后面 5.6 节）。

② 注：运输层传送的协议数据单元叫作报文段，网络层传送的协议数据单元叫作 **IP** 数据报。但在一般讨论问题时，都可把它们简称为分组。

就再发送下一个分组 M_2 。同样，在收到 B 对 M_2 的确认后，再发送 M_3 。

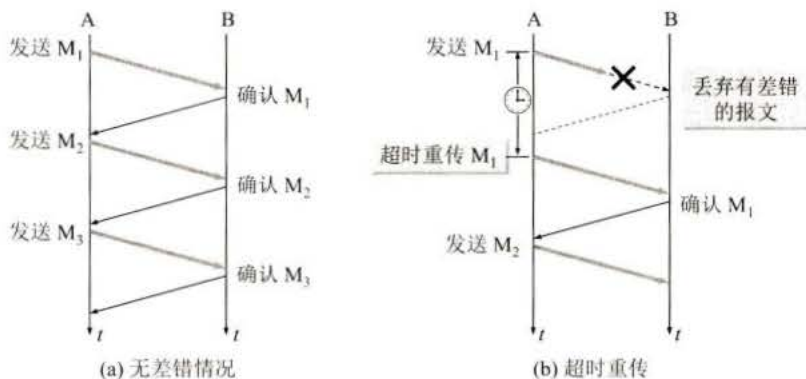


图 5-8 停止等待协议

2. 出现差错

图 5-8(b)是分组在传输过程中出现差错的情况。B 接收 M_1 时检测出了差错，就丢弃 M_1 ，其他什么也不做（不通知 A 收到有差错的分组）^①。也可能是 M_1 在传输过程中丢失了，这时 B 当然什么都不知道。在这两种情况下，B 都不会发送任何信息。可靠传输协议是这样设计的：A 只要超过了一段时间仍然没有收到确认，就认为刚才发送的分组丢失了，因而重传前面发送过的分组。这就叫作**超时重传**。要实现超时重传，就要在每发送完一个分组时设置一个**超时时器**。如果在超时时器到期之前收到了对方的确认，就撤销已设置的超时时器。其实在图 5-8(a)中，A 为每一个已发送的分组都设置了一个超时时器。但 A 只要在超时时器到期之前收到了相应的确认，就撤销该超时时器。为简单起见，这些细节在图 5-8(a)中都省略了。

这里应注意以下三点。

第一，A 在发送完一个分组后，必须暂时保留已发送的分组的副本（在发生超时重传时使用）。只有在收到相应的确认后才能清除暂时保留的分组副本。

第二，分组和确认分组都必须进行**编号**^②。这样才能明确是哪一个发送出去的分组收到了确认，而哪一个分组还没有收到确认。

第三，超时时器设置的重传时间应当比数据在分组传输的平均往返时间更长一些。图 5-8(b)中的一段虚线表示如果 M_1 正确到达 B 同时 A 也正确收到确认的过程。可见重传时间应设定为比平均往返时间更长一些。显然，如果重传时间设定得很长，那么通信的效率就

^① 注：在可靠传输的协议中，也可以在检测出有差错时发送“否认报文”给对方。这样做的好处是能够让发送方及早知道出现了差错。不过由于这样处理会使协议复杂化，现在实用的可靠传输协议都不使用这种否认报文了。

^② 注：编号并不是一个非常简单的问题。分组编号使用的位数总是有限的，同一个号码会重复使用。例如，10 位的编号范围是 0 ~ 1023。当编号增加到 1023 时，再增加一个号就又回到 0，然后重复使用这些号码。我们的家用电表、水表，以及汽车中的里程表，都有类似的问题。因此，在所发送的分组中，必须能够区分开哪些是新发送的，哪些是重传的。对于简单链路上传送的帧，如采用停止等待协议，只要用 1 位编号即可，也就是发送完 0 号帧，收到确认后，再发送 1 号帧，收到确认后，再发送 0 号帧。但是在运输层，这种编号方法有时并不能保证可靠传输（见习题 5-18）。

会很低。但如果重传时间设定得太短，以致产生不必要的重传，就浪费了网络资源。然而，在运输层重传时间的准确设定是非常复杂的，这是因为已发送出的分组到底会经过哪些网络，以及这些网络将会产生多大的时延（这取决于这些网络当时的拥塞情况），这些都是不确定因素。图 5-9 中把往返时间当作固定的（这显然不符合网络的实际情况），只是为了讲述原理的方便。关于重传时间应如何选择，在后面的 5.6.3 节还要进一步讨论。

3. 确认丢失和确认迟到

图 5-9(a)说明的是另一种情况。B 所发送的对 M_1 的确认丢失了。A 在设定的超时重传时间内没有收到确认，并无法知道是自己发送的分组出错、丢失，或者是 B 发送的确认丢失了。因此 A 在超时计时器到期后就要重传 M_1 。现在应注意 B 的动作。假定 B 又收到了重传的分组 M_1 。这时应采取两个行动。

第一，丢弃这个重复的分组 M_1 ，不向上层重复交付。

第二，向 A 发送确认。不能认为已经发送过确认就不再发送，因为 A 之所以重传 M_1 就表示 A 没有收到对 M_1 的确认。

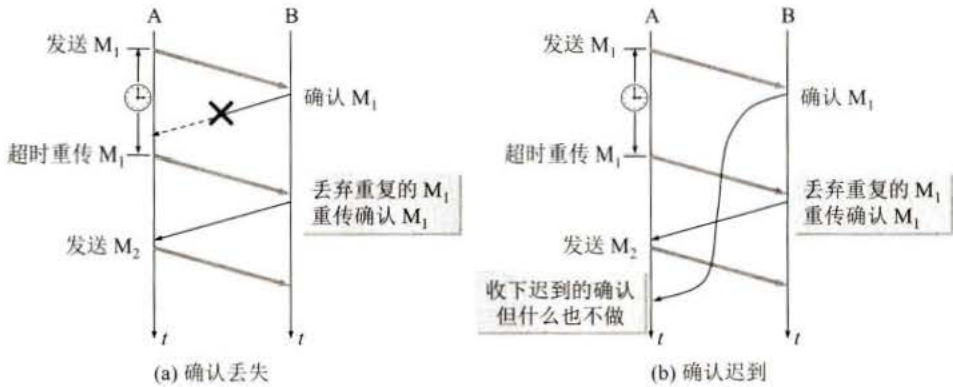


图 5-9 确认丢失和确认迟到

图 5-9(b)也是一种可能出现的情况。传输过程中没有出现差错，但 B 对分组 M_1 的确认迟到了。A 会收到重复的确认。对重复的确认的处理很简单：收下后就丢弃，但什么也不做。B 仍然会收到重复的 M_1 ，并且同样要丢弃重复的 M_1 ，并重传确认分组。

通常 A 最终总是可以收到对所有发出的分组的确认。如果 A 不断重传分组但总是收不到确认，就说明通信线路太差，不能进行通信。

使用上述的确认和重传机制，我们就可以在不可靠的传输网络上实现可靠的通信。

像上述的这种可靠传输协议常称为自动重传请求 ARQ (Automatic Repeat reQuest)。意思是重传的请求是自动进行的，因此也可见到自动请求重传这样的译名。接收方不需要请求发送方重传某个出错的分组。

4. 信道利用率

停止等待协议的优点是简单，但缺点是信道利用率太低。我们可以用图 5-10 来说明这个问题。为简单起见，假定在 A 和 B 之间有一条直通的信道来传送分组。

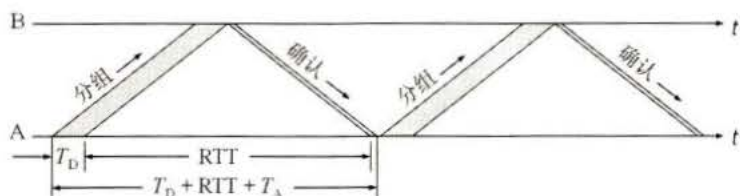


图 5-10 停止等待协议的信道利用率太低

假定 A 发送分组需要的时间是 T_D 。显然, T_D 等于分组长度除以数据率。再假定分组正确到达 B 后, B 处理分组的时间可以忽略不计, 同时立即发回确认。假定 B 发送确认分组需要时间 T_A 。如果 A 处理确认分组的时间也可以忽略不计, 那么 A 在经过时间 $(T_D + RTT + T_A)$ 后就可以再发送下一个分组, 这里的 RTT 是往返时间。因为仅仅是在时间 T_D 内才用来传送有用的数据 (包括分组的首部), 因此信道的利用率 U 可用下式计算:

$$U = \frac{T_D}{T_D + RTT + T_A} \quad (5-3)$$

请注意, 更细致的计算还可以在上式分子的时间 T_D 内扣除传送控制信息 (如首部) 所花费的时间。但在进行粗略计算时, 用近似的式(5-3)就可以了。

我们知道, 式(5-3)中的往返时间 RTT 取决于所使用的信道。例如, 假定 1200 km 的信道的往返时间 $RTT = 20 \text{ ms}$, 分组长度是 1200 bit, 发送速率是 1 Mbit/s。若忽略处理时间和 T_A (T_A 一般都远小于 T_D), 则可算出信道的利用率 $U = 5.66\%$ 。但若把发送速率提高到 10 Mbit/s, 则 $U = 5.96 \times 10^{-3}$ 。信道在绝大多数时间内都是空闲的。

从图 5-10 还可看出, 当往返时间 RTT 远大于分组发送时间 T_D 时, 信道的利用率就会非常低。还应注意的是, 图 5-10 并没有考虑出现差错后的分组重传。若出现重传, 则对传送有用的数据信息来说, 信道的利用率就还要降低。

为了提高传输效率, 发送方可以不使用低效率的停止等待协议, 而是采用流水线传输 (如图 5-11 所示)。流水线传输就是发送方可连续发送多个分组, 不必每发完一个分组就停顿下来等待对方的确认。这样可使信道上一直有数据在不间断地传送。显然, 这种传输方式可以获得很高的信道利用率。



图 5-11 流水线传输可提高信道利用率

当使用流水线传输时, 就要使用下面介绍的连续 ARQ 协议和滑动窗口协议。

5.4.2 连续 ARQ 协议

滑动窗口协议比较复杂, 是 TCP 协议的精髓所在。这里先给出连续 ARQ 协议最基本的概念, 但不涉及许多细节问题。详细的滑动窗口协议将在后面的 5.6 节中讨论。

图 5-12(a)表示发送方维持的发送窗口, 它的意义是: 位于发送窗口内的 5 个分组都可

连续发送出去，而不需要等待对方的确认。这样，信道利用率就提高了。

在讨论滑动窗口时，我们应当注意到，图中还有一个时间坐标（但以后往往省略这样的时间坐标）。按照习惯，“向前”是指“向着时间增大的方向”，而“向后”则是“向着时间减少的方向”。分组发送是按照分组序号从小到大发送的。



图 5-12 连续 ARQ 协议的工作原理

连续 ARQ 协议规定，发送方每收到一个确认，就把发送窗口向前滑动一个分组的位置。图 5-12(b)表示发送方收到了对第 1 个分组的确认，于是把发送窗口向前移动一个分组的位置。如果原来已经发送了前 5 个分组，那么现在就可以发送窗口内的第 6 个分组了。

接收方一般都是采用累积确认的方式。这就是说，接收方不必对收到的分组逐个发送确认，而是在收到几个分组后，对按序到达的最后一个分组发送确认，这就表示：到这个分组为止的所有分组都已正确收到了。

累积确认有优点也有缺点。优点是容易实现，即使确认丢失也不必重传；但缺点是不能向发送方及时反映接收方已经正确收到所有分组的信息。

例如，如果发送方发送了前 5 个分组，而中间的第 3 个分组丢失了。这时接收方只能对前两个分组发出确认。发送方无法知道后面三个分组的下落，而只好把后面的三个分组都再重传一次。这就叫作 Go-back-N（回退 N），表示需要再退回来重传已发送过的 N 个分组。可见当通信线路质量不好时，连续 ARQ 协议会带来负面的影响。

在深入讨论 TCP 的可靠传输问题之前，必须先了解 TCP 的报文段首部的格式。

5.5 TCP 报文段的首部格式

TCP 虽然是面向字节流的，但 TCP 传送的数据单元却是报文段。一个 TCP 报文段分为首部和数据两部分，而 TCP 的全部功能都体现在它首部中各字段的作用。因此，只有弄清 TCP 首部各字段的作用才能掌握 TCP 的工作原理。下面讨论 TCP 报文段的首部格式。

TCP 报文段首部的前 20 个字节是固定的（如图 5-13 所示），后面有 $4n$ 字节是根据需要而增加的选项（ n 是整数）。因此 TCP 首部的最小长度是 20 字节。

首部固定部分各字段的意义如下：

(1) **源端口和目的端口** 各占 2 个字节，分别写入源端口号和目的端口号。和前面图 5-5 所示的 UDP 的分用相似，TCP 的分用功能也是通过端口实现的。

(2) **序号** 占 4 字节。序号范围是 $[0, 2^{32} - 1]$ ，共 2^{32} （即 4 294 967 296）个序号。序号增加到 $2^{32} - 1$ 后，下一个序号就又回到 0。也就是说，序号使用 $\text{mod } 2^{32}$ 运算。TCP 是面

扫一扫



视频讲解

向字节流的。在一个 TCP 连接中传送的字节流中的每一个字节都按顺序编号。整个要传送的字节流的起始序号必须在连接建立时设置。首部中的序号字段值则指的是本报文段所发送的数据的第一个字节的序号。例如，一报文段的序号字段值是 301，而携带的数据共有 100 字节。这就表明：本报文段的数据的第一个字节的序号是 301，最后一个字节的序号是 400。显然，下一个报文段（如果还有的话）的数据序号应当从 401 开始，即下一个报文段的序号字段值应为 401。这个字段的名称也叫作“报文段序号”。

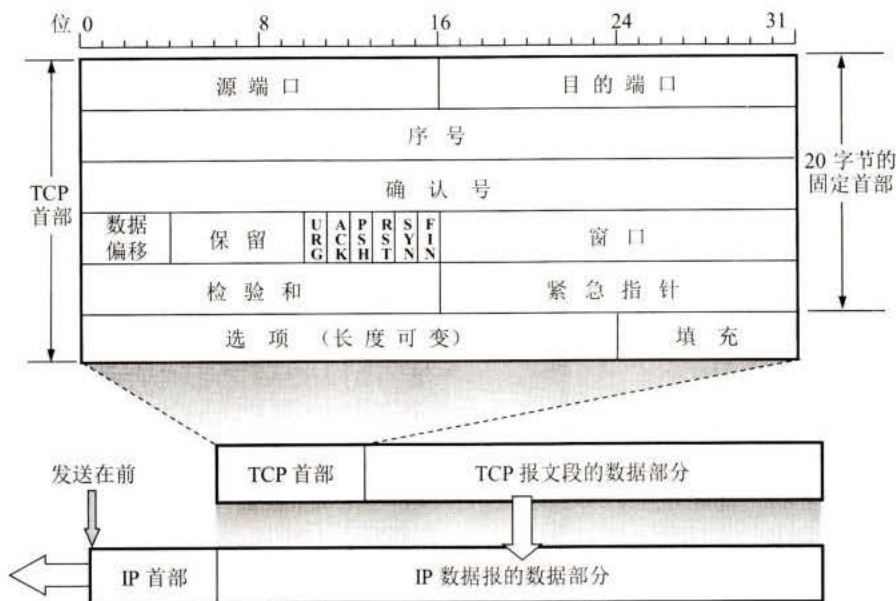


图 5-13 TCP 报文段的首部格式

(3) 确认号 占 4 字节，是期望收到对方下一个报文段的第一个数据字节的序号。例如，B 正确收到了 A 发送过来的一个报文段，其序号字段值是 501，而数据长度是 200 字节（序号 501 ~ 700），这表明 B 正确收到了 A 发送的到序号 700 为止的数据。因此，B 期望收到 A 的下一个数据序号是 701，于是 B 在发送给 A 的确认报文段中把确认号置为 701。请注意，现在的确认号不是 501，也不是 700，而是 701。

总之，应当记住：

若确认号 = N ，则表明：到序号 $N-1$ 为止的所有数据都已正确收到。

由于序号字段有 32 位长，可对 4 GB（即 4 千兆字节）的数据进行编号。在一般情况下可保证当序号重复使用时，旧序号的数据早已通过网络到达终点了。

(4) 数据偏移 占 4 位，它指出 TCP 报文段的数据起始处距离 TCP 报文段的起始处有多远。这个字段实际上是指出 TCP 报文段的首部长度。由于首部中还有长度不确定的选项字段，因此数据偏移字段是必要的。但应注意，“数据偏移”的单位是 32 位字（即以 4 字节长的字为计算单位）。由于 4 位二进制数能够表示的最大十进制数字是 15，因此数据偏移的最大值是 60 字节，这也是 TCP 首部的最大长度（即选项长度不能超过 40 字节）。

(5) 保留 占 6 位，保留为今后使用，但目前应置为 0。

下面有 6 个控制位，用来说明本报文段的性质，它们的意义见下面的(6)~(11)。

(6) 紧急 URG (URGent) 当 $URG = 1$ 时，表明紧急指针字段有效。它告诉系统此报

文段中有紧急数据，应尽快传送（相当于高优先级的数据），而不要按原来的排队顺序传送。例如，已经发送了很长的一个程序要在远地的主机上运行。但后来发现了一些问题，需要取消该程序的运行。因此用户从键盘发出中断命令（Control-C）。如果不使用紧急数据，那么这两个字符将存储在接收 TCP 的缓存末尾。只有在所有的数据被处理完毕后这两个字符才被交付接收方的应用进程。这样做就浪费了许多时间。

当 URG 置 1 时，发送应用进程就告诉发送方的 TCP 有紧急数据要传送。于是发送方 TCP 就把紧急数据插入到本报文段数据的最前面，而在紧急数据后面的数据仍是普通数据。这时要与首部中紧急指针(Urgent Pointer)字段配合使用。

然而在紧急指针字段的具体实现上，由于过去的有些文档有错误或有不太明确的地方，这就导致人们对有关的 RFC 文档产生了不同的理解。于是，2011 年公布的建议标准 RFC 6093，对紧急指针字段的使用方法做出了更加明确的解释，并更新了几个重要的 RFC 文档，如 RFC 793, RFC 1011, RFC 1122 等。

(7) 确认 ACK (ACKnowledgment) 仅当 ACK = 1 时确认号字段才有效。当 ACK = 0 时，确认号无效。TCP 规定，在连接建立后所有传送的报文段都必须把 ACK 置为 1。

(8) 推送 PSH (PuSH) 当两个应用进程进行交互式的通信时，有时在一端的应用进程希望在键入一个命令后立即就能够收到对方的响应。在这种情况下，TCP 就可以使用推送(push)操作。这时，发送方 TCP 把 PSH 置 1，并立即创建一个报文段发送出去。接收方 TCP 收到 PSH = 1 的报文段，就尽快地（即“推送”向前）交付接收应用进程，而不再等到整个缓存都填满了后再向上交付。

虽然应用程序可以选择推送操作，但推送操作很少使用。

(9) 复位 RST (ReSeT) 当 RST = 1 时，表明 TCP 连接中出现严重差错（如主机崩溃或其他原因），必须释放连接，然后再重新建立运输连接。将 RST 置为 1 还用来拒绝一个非法的报文段或拒绝打开一个连接。RST 也可称为重建位或重置位。

(10) 同步 SYN (SYNchronization) 在连接建立时用来同步序号。当 SYN = 1 而 ACK = 0 时，表明这是一个连接请求报文段。对方若同意建立连接，则应在响应的报文段中使 SYN = 1 和 ACK = 1。因此，SYN 置为 1 就表示这是一个连接请求或连接接受报文。关于连接的建立和释放，在后面的 5.9 节还要进行详细讨论。

(11) 终止 FIN (FINish，意思是“完了”“终止”) 用来释放一个连接。当 FIN = 1 时，表明此报文段的发送方的数据已发送完毕，并要求释放运输连接。

(12) 窗口 占 2 字节。窗口值是 $[0, 2^{16} - 1]$ 之间的整数。窗口指的是发送本报文段的一方的接收窗口（而不是自己的发送窗口）。窗口值告诉对方：从本报文段首部中的确认号算起，接收方目前允许对方发送的数据量（以字节为单位）。之所以要有这个限制，是因为接收方的数据缓存空间是有限的。总之，窗口值作为接收方让发送方设置其发送窗口的依据。

例如，发送了一个报文段，其确认号是 701，窗口字段是 1000。这就是告诉对方：“从 701 号算起，我（即发送此报文段的一方）的接收缓存空间还可接收 1000 个字节数据（字节序号是 701 ~ 1700），你在给我发送数据时，必须考虑到我的接收缓存容量。”

总之，应当记住：

窗口字段明确指出了现在允许对方发送的数据量。窗口值经常在动态变化着。

扫一扫



视频讲解

(13) **检验和** 占 2 字节。检验和字段检验的范围包括首部和数据这两部分。和 UDP 用户数据报一样，在计算检验和时，要在 TCP 报文段的前面加上 12 字节的伪首部。伪首部的格式与图 5-5 中 UDP 用户数据报的伪首部一样。但应把伪首部第 4 个字段中的 17 改为 6（TCP 的协议号是 6），把第 5 字段中的 UDP 长度改为 TCP 长度。接收方收到此报文段后，仍要加上这个伪首部来计算检验和。若使用 IPv6，则相应的伪首部也要改变。

(14) **紧急指针** 占 2 字节。紧急指针仅在 $URG = 1$ 时才有意义，它指出本报文段中的紧急数据的字节数（紧急数据结束后就是普通数据）。因此，紧急指针指出了紧急数据的末尾在报文段中的位置。当所有紧急数据都处理完时，TCP 就告诉应用程序恢复到正常操作。值得注意的是，即使窗口为零时也可发送紧急数据。

(15) **选项** 长度可变，最长可达 40 字节。当没有使用“选项”时，TCP 的首部长度是 20 字节。最后的填充字段仅仅是为了使整个 TCP 首部长度是 4 字节的整数倍。

TCP 最初只规定了一种选项，即**最大报文段长度 MSS** (Maximum Segment Size) [RFC 6691]。请注意 MSS 这个名词的含义。MSS 是**每一个 TCP 报文段中的数据字段的最大长度**。数据字段加上 TCP 首部才等于整个的 TCP 报文段。所以 MSS 并不是整个 TCP 报文段的最大长度，而是“TCP 报文段长度减去 TCP 首部长度”。

为什么要规定一个最大报文段长度 MSS 呢？这并不是考虑接收方的接收缓存可能放不下 TCP 报文段中的数据。实际上，MSS 与接收窗口值没有关系。我们知道，TCP 报文段的数据部分，至少要加上 40 字节的首部（TCP 首部 20 字节和 IP 首部 20 字节，这里都还没有考虑首部中的选项部分），才能组装成一个 IP 数据报。若选择较小的 MSS 长度，网络的利用率就降低了。设想在极端的情况下，当 TCP 报文段只含有 1 字节的数据时，在 IP 层传输的数据报的开销至少有 40 字节（包括 TCP 报文段的首部和 IP 数据报的首部）。这样，对网络的利用率就不会超过 $1/41$ 。到了数据链路层还要加上一些开销。但反过来，若 TCP 报文段非常长，那么在 IP 层传输时就有可能要分解成多个短数据报片。在终点要把收到的各个短数据报片装配成原来的 TCP 报文段。当传输出错时还要进行重传。这些也都会使开销增大。

因此，从提高网络传输效率考虑，MSS 应尽可能大些，只要在 IP 层传输时不需要再分片就行。由于 IP 数据报所经历的路径是动态变化的，因此在某条路径上确定的不需要分片的 MSS，如果改走另一条路径就可能需要进行分片。因此最佳的 MSS 实际上是很难确定的。在连接建立的过程中，双方都把自己能够支持的 MSS 写入这一字段，以后就按照这个数值传送数据，两个传送方向可以有不同的 MSS 值^①。若主机未填写这一项，则 MSS 的默认值是 536 字节（这个数值来自 576 字节的 IP 数据报总长度减去 TCP 和 IP 的固定首部）。因此，所有互联网上的主机都应能接受的报文段长度是 $536 + 20$ （固定首部长度）= 556 字节。

随着互联网的发展，又陆续增加了几个选项，如**窗口扩大选项**、**时间戳选项**等（见建议标准 RFC 7323）。以后又增加了有关**选择确认(SACK)**选项（见建议标准 RFC 2018）。这些选项的位置都在图 5-13 所示的选项字段中。

窗口扩大选项是为了扩大窗口。我们知道，TCP 首部中窗口字段长度是 16 位，因此最大的窗口大小为 64 KB（见下一节）。虽然这对早期的网络是足够用的，但对于包含卫星信

^① 注：流行的一种说法，在 TCP 连接建立阶段“双方协商 MSS 值”，但这是错误的，因为这里并不存在任何的协商，而只是一方把 MSS 值设定好以后再通知另一方而已。

道的网络^①，其传播时延和带宽都很大，要获得高吞吐率需要更大的窗口大小。

窗口扩大选项占 3 字节，其中有一个字节表示移位值 S。新的窗口值等于 TCP 首部中的窗口位数从 16 增大到 $(16 + S)$ 。移位值允许使用的最大值是 14，相当于窗口最大值增大到 $2^{(16+14)} - 1 = 2^{30} - 1$ 。

窗口扩大选项可以在双方初始建立 TCP 连接时进行协商。如果连接的某一端实现了窗口扩大，当它不再需要扩大其窗口时，可发送 $S = 0$ 的选项，使窗口大小回到 16。

时间戳选项占 10 字节，其中最主要的字段是时间戳值字段（4 字节）和时间戳回送回答字段（4 字节）。时间戳选项有以下两个功能：

第一，用来计算往返时间 RTT（见后面的 5.6.2 节）。发送方在发送报文段时把当前时钟的时间值放入时间戳字段，接收方在确认该报文段时把时间戳字段值复制到时间戳回送回答字段。因此，发送方在收到确认报文后，可以准确地计算出 RTT。

第二，用于处理 TCP 序号超过 2^{32} 的情况，这又称为防止序号绕回 PAWS (Protect Against Wrapped Sequence numbers)。我们知道，TCP 报文段的序号只有 32 位，而每增加 2^{32} 个序号就会重复使用原来用过的序号。当使用高速网络时，在一次 TCP 连接的数据传送中序号很可能会被重复使用。例如，当使用 1.5 Mbit/s 的速率发送报文段时，序号重复要 6 小时以上。但若用 2.5 Gbit/s 的速率发送报文段，则不到 14 秒序号就会重复。为了使接收方能够把新的报文段和迟到很久的报文段区分开，可以在报文段中加上这种时间戳。

我们将在后面的 5.6.3 节介绍选择确认选项。

5.6 TCP 可靠传输的实现

本节讨论 TCP 可靠传输的实现。

我们首先介绍以字节为单位的滑动窗口。为了讲述可靠传输原理的方便，我们假定数据传输只在一个方向进行，即 A 发送数据，B 给出确认。这样的好处是使讨论限于两个窗口，即发送方 A 的发送窗口和接收方 B 的接收窗口。如果再考虑 B 也向 A 发送数据，那么还要增加 A 的接收窗口和 B 的发送窗口，这样总共有 4 个都不断在变化大小的窗口。这对讲述可靠传输的原理并没有多少帮助，反而会使问题变得更加烦琐。

5.6.1 以字节为单位的滑动窗口

TCP 的滑动窗口是以字节为单位的。为了便于说明滑动窗口的工作原理，我们故意把后面图 5-14 至图 5-17 中的字节编号都取得很小（实际的窗口大小多为数千字节）。现假定 A 收到了 B 发来的确认报文段，其中窗口是 20 字节，而确认号是 31（这表明 B 期望收到的下一个字节序号是 31（请注意，这里不是分组的序号），而到序号 30 为止的数据已经收到了）。根据这两个数据，A 就构造出自己的发送窗口，如图 5-14 所示。



^① 注：这种信道常称为长粗管道(long fat pipe)。